

CEBU INSTITUTE OF TECHNOLOGY - UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Design Description

For

RigMD: A Symptom-Guided Diagnostic and Controlled Remediation Support System for Non-Technical Windows Desktop PC Users

Team Code: 2526-sem2-it332-11

Document Version: 1.2 Repository-Aligned Final Draft

Revision Date: 05 September 2026

Prepared by:

Libato, Michael Grant

Macansantos, Axcelson

Ruperez, Raymart

Maestrado, Ralph Keane

Labaya, Godwin

Change History

Version	Date	Description	Author(s)
1.0	22 May 2026	Initial release of Software Design Description for RigMD.	Team 2526-sem2-it332-11
1.1	30 August 2026	Format-preserved revision aligned with GitHub	Team 2526-sem2-it332-11
1.2	05 September 2026	Updated newest Agent command flow, network telemetry/rules, Agent-backed remediation, hybrid persistence, installer design, and AI provider accuracy	Team 2526-sem2-it332-11

Signature

Name	Signature / Date
Libato, Michael Grant	
Macansantos, Axcelson	
Ruperez, Raymart	
Maestrado, Ralph Keane	
Labaya, Godwin	

Preface

This Software Design Description (SDD) presents the updated architectural and detailed design of RigMD - A Symptom-Guided Diagnostic and Controlled Remediation Support System for Non-Technical Windows Desktop PC Users. The document was prepared by Team 2526-sem2-it332-11 of the College of Computer Studies at Cebu Institute of Technology University in partial fulfillment of the requirements for IT332.

RigMD is a Windows-focused diagnostic system that assists individual desktop PC owners in identifying probable internal causes of Windows-observable symptoms and deciding on an

appropriate, clearly classified next action. The current implementation integrates React-based screens, an ASP.NET Core backend, C# diagnostic rules, Windows/WMI telemetry, SQLite persistence, constrained AI explanations, a Windows Agent for fresh evidence collection, and controlled remediation with safety checks.

This document is intended for use by the development team during implementation, by faculty evaluators during design review, and by future developers who may extend or maintain the system. This revision preserves the original SDD module, table format, and embedded images while updating the contents to match the latest GitHub main branch at commit f1a4a47.

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions and Acronyms

1.4 References

2. Architectural Design

3. Detailed Design

Current architecture note: the embedded diagrams/images are preserved from the previous SDD as requested. The intended diagram content should show React/Vite frontend, ASP.NET Core API, C# Application and Domain services, SQLite local session/remediation persistence, Supabase-backed Agent devices/commands/snapshots, Windows Agent service, Windows/WMI/network providers, remediation safety policy, and the Inno Setup deployment path.

Module 1 - System Profile and System Overview Module

1.1 Home Dashboard and Quick Actions

1.2 System Profile Detection

Module 2 - Structured OS Symptom Intake Module

2.1 Guided Diagnostic Intake

Module 3 - Internal Diagnostic Interpretation Engine

3.1 Diagnostic Result Summary

3.2 Supporting Diagnostic Details

3.3 Recommended Actions and Decision Confidence

Module 4 - Advisory Action Module

4.1 Warning Signs Reference

Module 5 - Transparent Result Dashboard and Report Output Module

5.1 Reports

Module 6 - Session History and Recurring Pattern Detection Module

6.1 Diagnostic History

6.2 Recurring Patterns

1. Introduction

1.1 Purpose

This Software Design Description defines the architecture and detailed design of RigMD, a symptom-guided diagnostic and controlled remediation support system for Windows desktop PC users. The document specifies system components, module boundaries, transaction flows, user interface responsibilities, frontend and backend components, object-oriented design references, data design, and safety boundaries. It serves as the primary technical reference for development, integration, evaluation, and future maintenance.

RigMD helps individual non-technical Windows desktop PC owners interpret Windows-observable symptoms and arrive at a clear, action-classified decision about their next troubleshooting step. The system guides users from hardware/profile observation to structured or automatic diagnosis, diagnostic interpretation, advisory output, controlled remediation where allowed, resolution rechecking, and history review.

1.2 Scope

This document covers the design of the RigMD desktop application as scoped in the approved SRS. The application targets individual Windows desktop PC users, including students, gamers, beginner PC builders, budget desktop owners, content creators, and general home users who experience Windows-observable performance symptoms but lack the technical background to diagnose them independently.

The system is limited to functional or partially functional Windows desktop PCs. Fully non-functional systems are handled only through a boot-status triage path and are routed to a professional inspection recommendation. The system does not perform physical hardware testing, voltage measurement, component substitution, or direct replacement and upgrade recommendations.

The current application is designed around a React + Vite frontend and a C#/.NET ASP.NET Core backend. It uses C# domain and application services for diagnostic logic, Windows/WMI providers and a Windows Agent for telemetry, SQLite through Entity Framework Core for local application/session persistence, Supabase PostgreSQL for Agent device, command, and snapshot communication, and Gemini or offline fallback services for constrained plain-language explanation generation.

1.3 Definitions and Acronyms

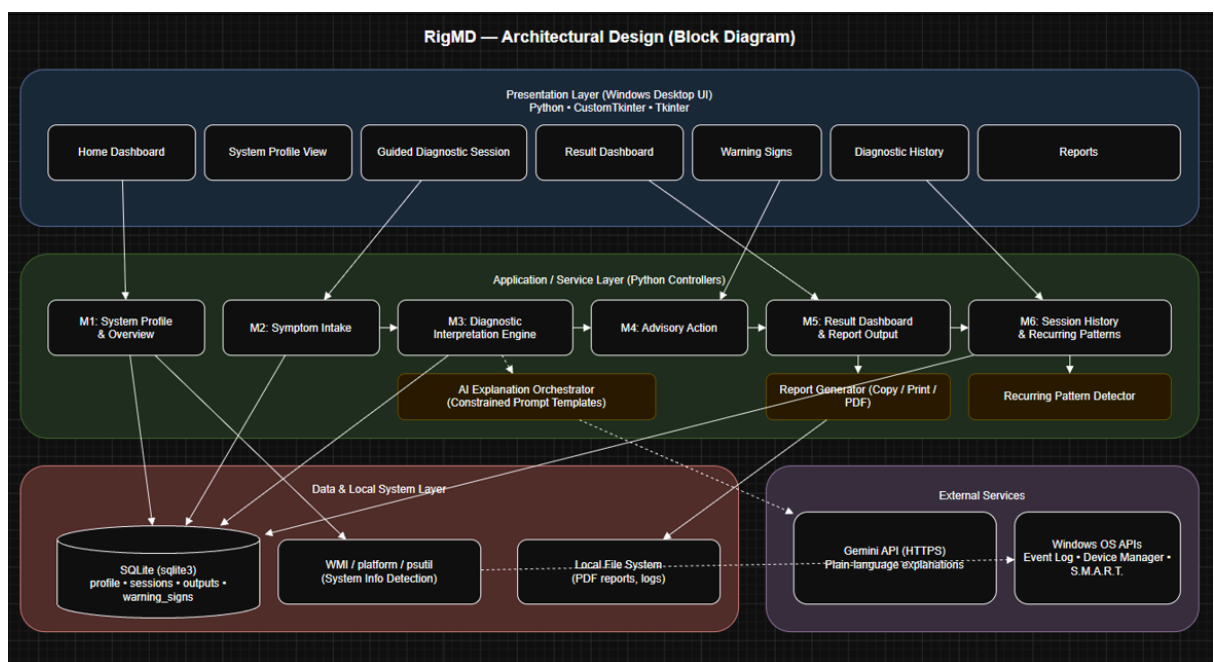
Term / Acronym	Definition
RigMD	A symptom-guided Windows PC diagnostic and controlled remediation support system for non-technical users.
SDD	Software Design Description - this document.
SRS	Software Requirements Specification - the requirements document this SDD is derived from.
GUI	Graphical User Interface - the presentation layer currently implemented with React and Vite.
API	Application Programming Interface exposed by the ASP.NET Core backend.
Gemini API	The current AI explanation provider used by GeminiAiExplainer with OfflineAiExplainer fallback.
SQLite	Local database used for application/session persistence through Entity Framework Core.
Supabase	PostgreSQL service currently used for Agent devices, Agent commands, Agent snapshots, and legacy/optional synchronization concerns.
WMI	Windows Management Instrumentation - Windows interface used for local system information retrieval.
Windows Agent	Installed background service that claims allowlisted commands, executes supported scans/remediations, and sends results/proof to the API.
DNS	Domain Name System; RigMD tests DNS resolution through WindowsNetworkProvider.
BSOD	Blue Screen of Death - Windows stop-error screen indicating critical system failure.

1.4 References

- RigMD Software Requirements Specification, Team 2526-sem2-it332-11, Semester 2, AY 2025-2026.
- RigMD Project Proposal (Weeks 7-8), Team 2526-sem2-it332-11.
- Kostopoulos, G., Davrazos, G., & Kotsiantis, S. (2024). Explainable artificial intelligence-based decision support systems: A recent review. *Electronics*, 13(14), 2842.
- Ravi, M., Negi, A., Bommi, N. S., & Rouf, N. (2025). Evolution of AI-driven decision making with decision support systems, expert systems, recommender systems, and XAI. *IETE Technical Review*, 42(4), 428-465.
- Brooke, J. (1996). SUS: A quick and dirty usability scale. In *Usability Evaluation in Industry* (pp. 189-194). Taylor and Francis.
- Python 3.x Documentation. <https://docs.python.org/3/>
- CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/>
- Google Gemini API Documentation. <https://ai.google.dev/>
- SQLite Documentation. <https://www.sqlite.org/docs.html>

2. Architectural Design

RigMD follows a Windows-focused layered architecture. The user interacts with a React + Vite presentation layer. The ASP.NET Core API exposes application services. The C# application and domain layers manage system profile detection, structured intake, automatic diagnosis, evidence-based rule mapping, advisory action classification, controlled remediation, resolution rechecking, history, and recurring pattern detection. The infrastructure layer handles Windows/WMI telemetry including network telemetry, Windows Agent command communication, SQLite persistence through EF Core, Supabase-backed Agent device/command/snapshot records, installer support, and constrained Gemini explanation generation.



Technology Stack Summary

Layer	Technology	Design Purpose
Presentation Layer	React + Vite + TypeScript	Provides dashboards, guided/automatic diagnosis, history, warning signs, and remediation controls.
Application Logic Layer	C# application/domain services	Handles symptom intake, evidence-based automatic diagnosis, diagnostic mapping, advisory classification, remediation planning, resolution rechecking, and recurring pattern logic.

Layer	Technology	Design Purpose
System Information Layer	C# WMI/System.Management providers, WindowsNetworkProvider, RigMD Windows Agent	Retrieves local/agent-sourced hardware, OS, process, storage-volume, utilization, active adapter, IPv4, gateway, DNS server, and DNS resolution telemetry.
Data Persistence Layer	SQLite via EF Core; Supabase PostgreSQL for Agent communication	Stores sessions/remediation locally and stores Agent devices, commands, snapshots, results, and command proof through Supabase-backed AgentRepository records.
AI Explanation Layer	GeminiAiExplainer with OfflineAiExplainer fallback	Generates constrained plain-language explanations from deterministic diagnostic outputs without controlling actions.
Deployment / Installer	PowerShell build script + Inno Setup	Publishes the Agent as win-x64 self-contained and builds RigMD-Setup.exe to install RigMDAgent as an auto-start Windows service.

3. Detailed Design

Each module below follows the original SDD structure: User Interface Design, Front-end Components, Back-end Components, Object-Oriented Components, Class Diagram, Sequence Diagram, and Data Design. The section order and table format are preserved from the draft, while the component names, technologies, responsibilities, and persistence descriptions are updated to reflect the latest React and C#/.NET implementation.

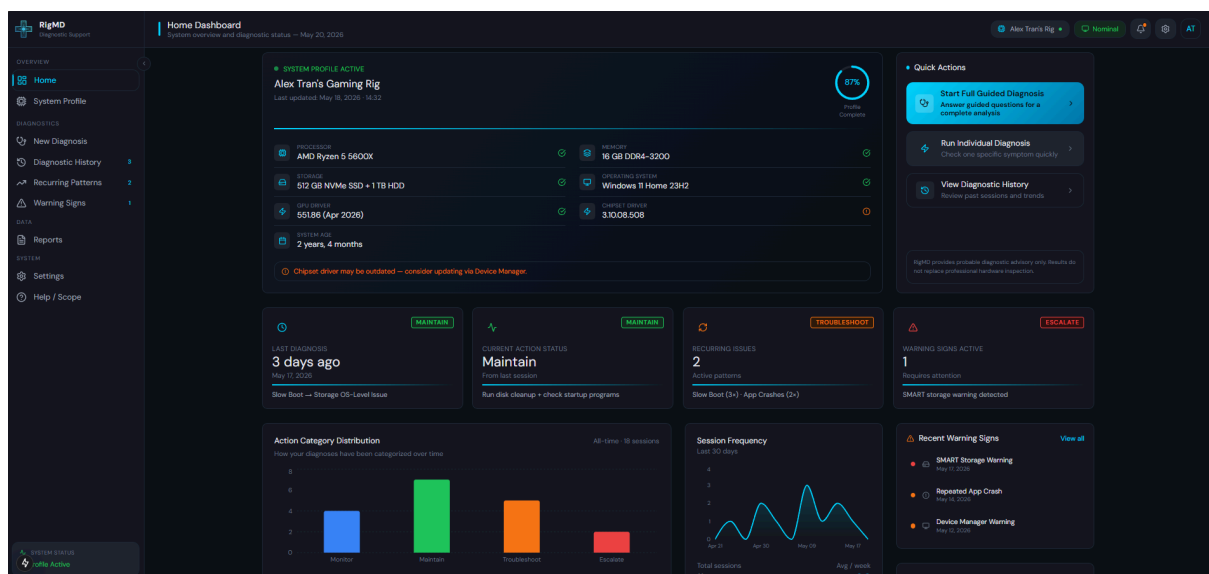
Module 1 - System Profile and System Overview Module

This module provides the first system overview and manages the desktop PC system profile used as diagnostic context. It includes the Hardware Dashboard, dashboard summary, system profile detection, local Windows telemetry, network telemetry, and the Windows Agent telemetry and command path.

1.1 Home Dashboard and Quick Actions

User Interface Design

The dashboard is the main overview screen shown after the application launches. It summarizes hardware telemetry, network state, latest diagnostic result, recurring issue count, warning signs, remediation availability, Agent status, and overall action status. The interface provides quick actions for starting guided diagnosis, requesting automatic evidence-based diagnosis, viewing diagnostic history, and checking warning indicators.



Front-end Components

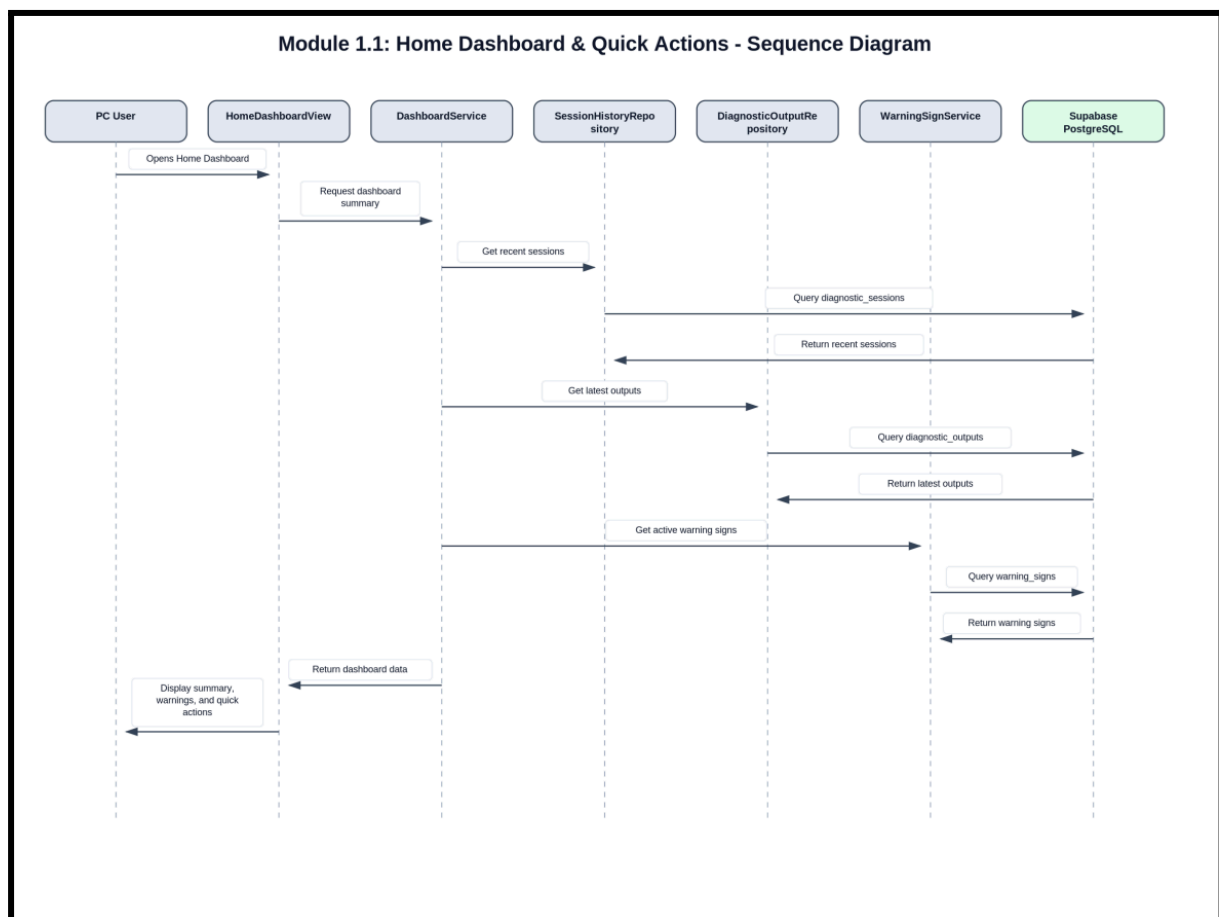
Component Name	Description and Purpose	Component Type / Format
HardwareDashboard	Displays current system telemetry, network/Agent status, latest diagnostic status, and quick entry points.	React Page
HomeDashboardView	Renders dashboard summary sections inside HardwareDashboard.	React Component
DashboardMetricCard	Shows session, recurring, warning, and escalation metrics.	React Component
RecentActivityCard	Displays recent diagnostic sessions and activity trends.	React Component
ActiveWarningSignsCard	Shows warning signs detected or recently connected to diagnostic results.	React Component

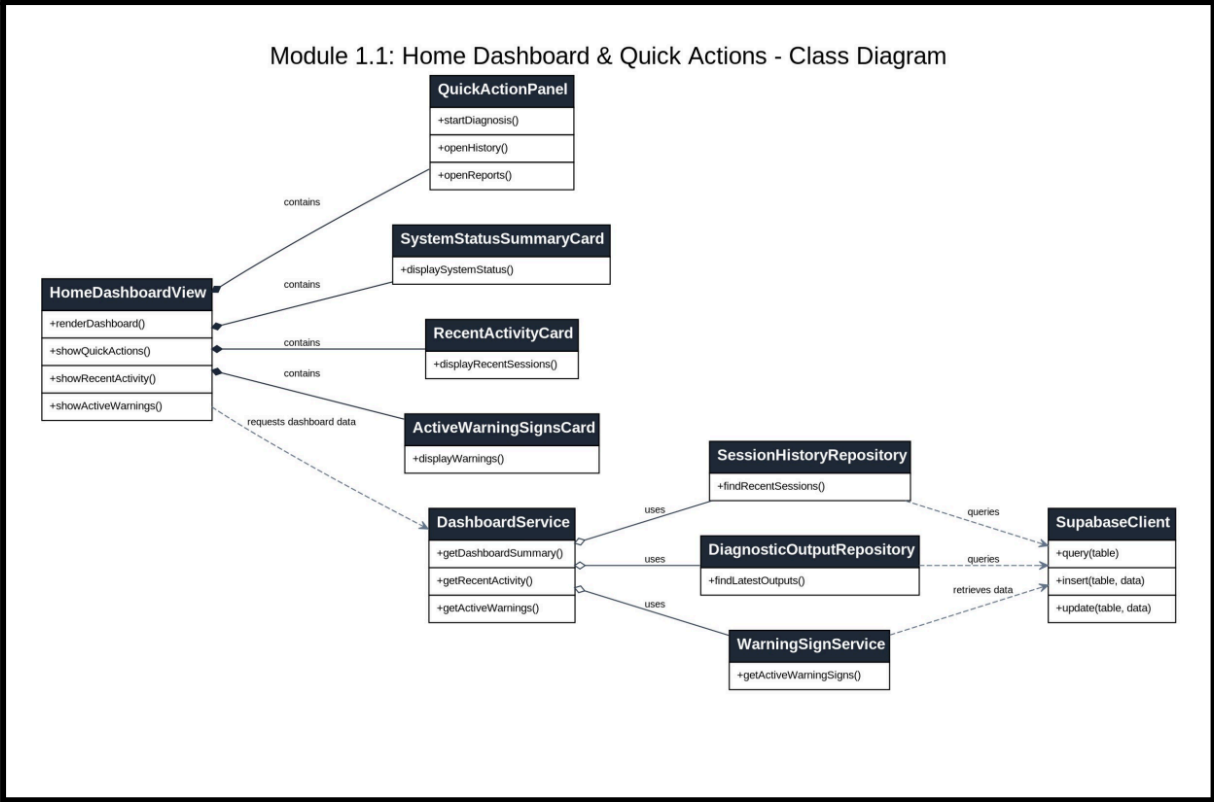
Back-end Components

Component Name	Description and Purpose	Component Type / Format
DashboardController	Exposes dashboard summary data through GET <code>/api/dashboard/summary</code> .	ASP.NET Core Controller

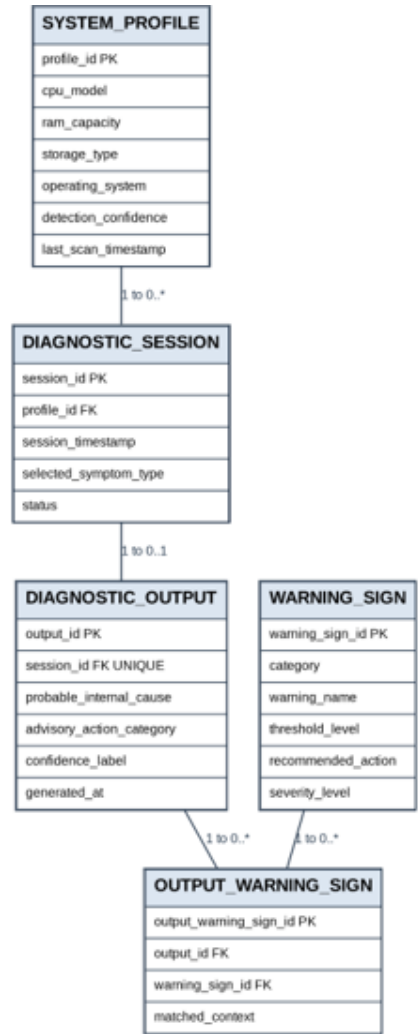
DiagnosticSessionRepository	Retrieves recent diagnostic sessions from local SQLite.	EF Core Repository
AgentRepository	Retrieves Agent device/snapshot status from Supabase for Agent-aware dashboard flows.	Npgsql/Supabase Repository
WarningSignService	Retrieves active or relevant warning signs for display.	C# Application Service

Object-Oriented Components





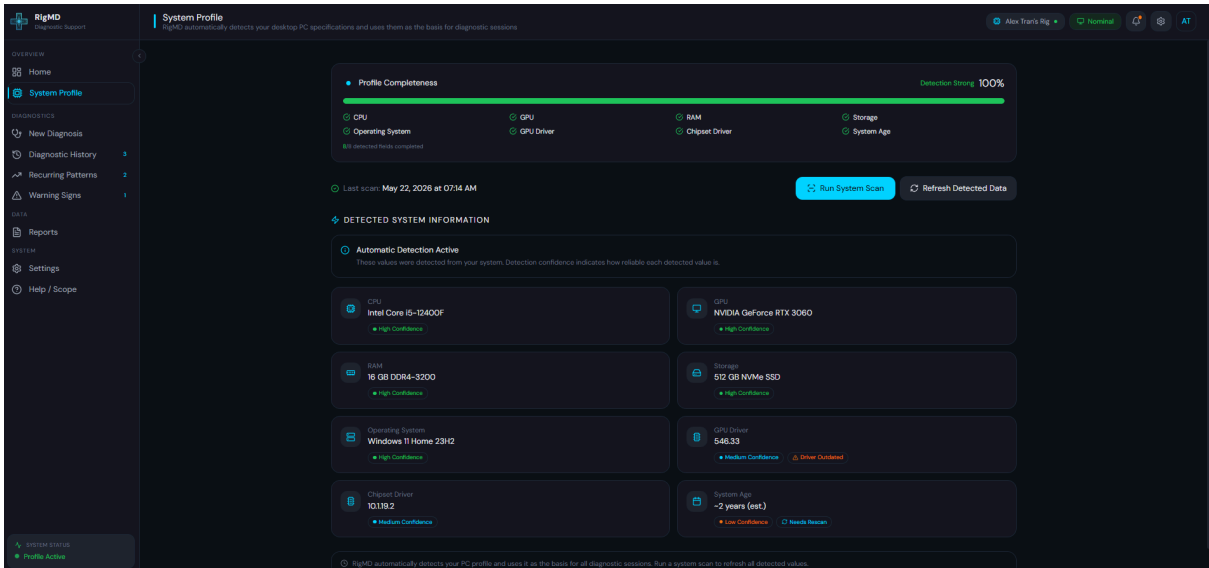
Data Design / ERD or Schema for Home Dashboard and Quick Actions



1.2 System Profile Detection

User Interface Design

The System Profile Detection screen displays hardware, software, and network information detected from the user's Windows desktop PC. The interface focuses on automatically detected data such as CPU, GPU, RAM, storage, logical volumes, operating system version, GPU driver version, motherboard/system metadata, running processes, utilization, active network adapter, IPv4 configuration, default gateway, DNS server configuration, DNS resolution status, scan status, and detection freshness. If detection is incomplete, stale, or unavailable, the interface marks the affected field and allows the user to refresh local telemetry or request a Windows Agent scan.



Front-end Components for System Profile Detection

Component Name	Description and Purpose	Component Type / Format
SystemProfileView	Displays CPU, RAM, storage, OS, GPU, driver, motherboard, utilization, process, and network details.	React Page

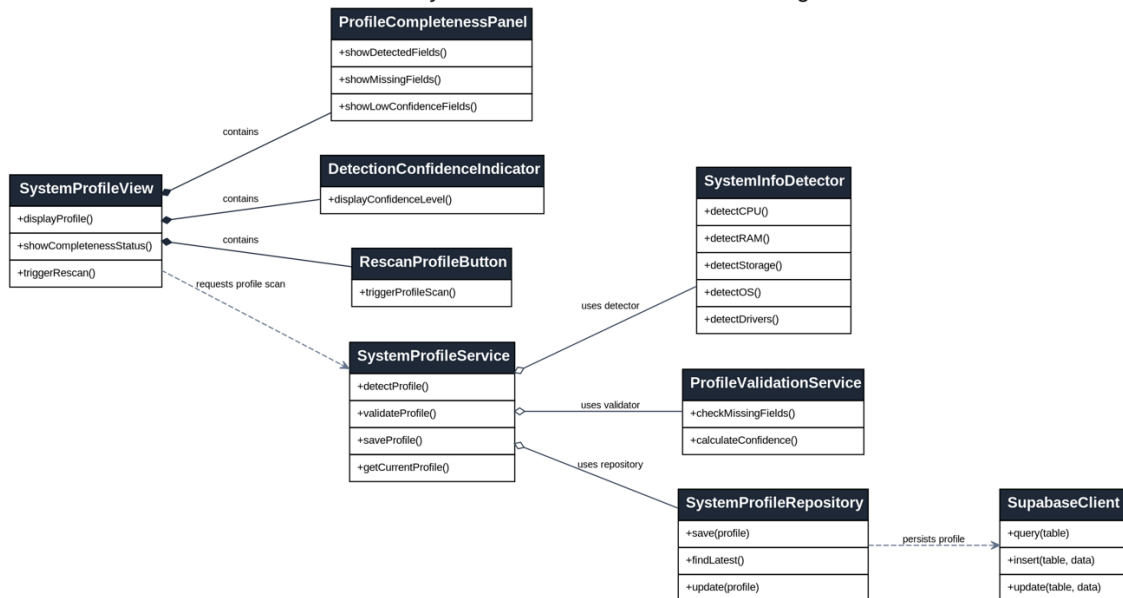
ProfileCompletenessPanel	Shows whether profile fields are detected, missing, stale, or agent-sourced.	React Component
RescanProfileButton	Allows the user to refresh hardware telemetry or request an Agent scan command.	React Button

Back-end Components for System Profile Detection

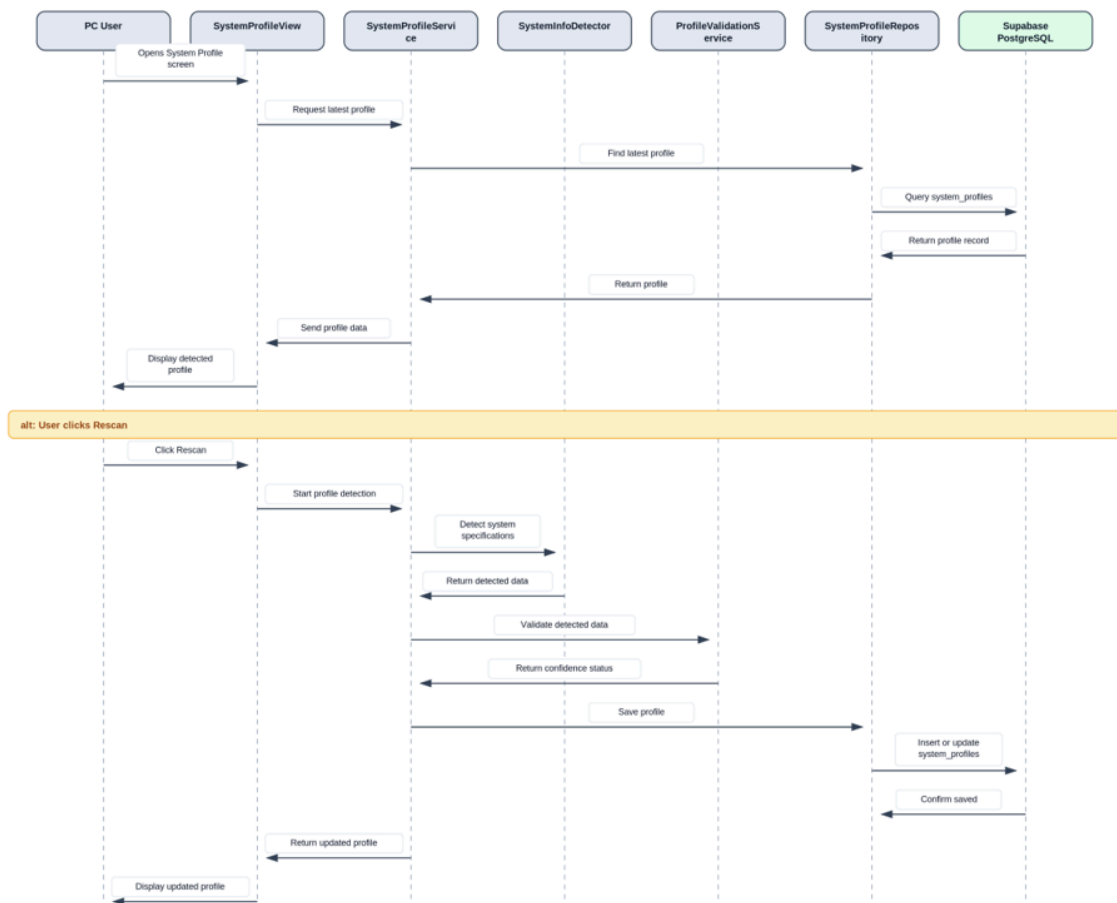
Component Name	Description and Purpose	Component Type / Format
WindowsSystemProfileService	Aggregates hardware, process, storage, and network providers into one HardwareProfileDto.	C# Infrastructure Service
WmiCpu/Gpu/Memory/Storage/OS Providers	Read local Windows hardware and OS information using WMI/System.Management.	C# Provider Classes
WindowsNetworkProvider	Detects active adapter, IPv4 address, default gateway, DNS servers, and example.com DNS resolution.	C# Infrastructure Provider
AgentController / AgentRepository	Queues Agent scan/remediation commands and stores Agent devices, snapshots, commands, result JSON, and proof through Supabase.	ASP.NET Core Controller + Npgsql Repository

Object-Oriented Components

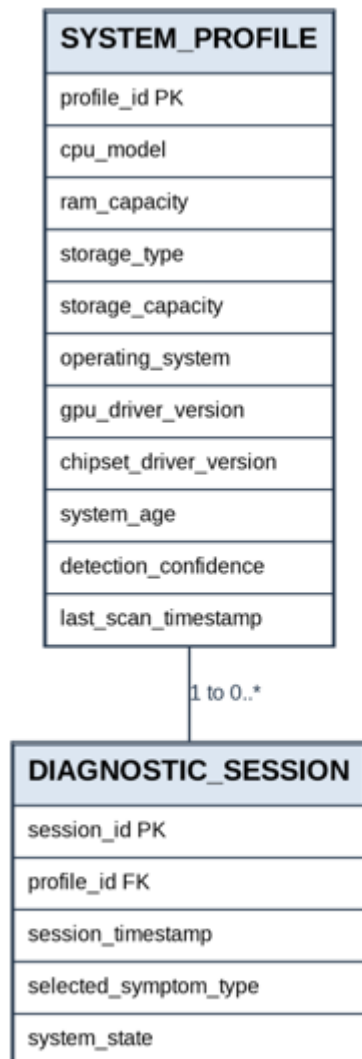
Module 1.2: System Profile Detection - Class Diagram



Module 1.2: System Profile Detection - Sequence Diagram



Data Design / ERD or Schema for System Profile Detection



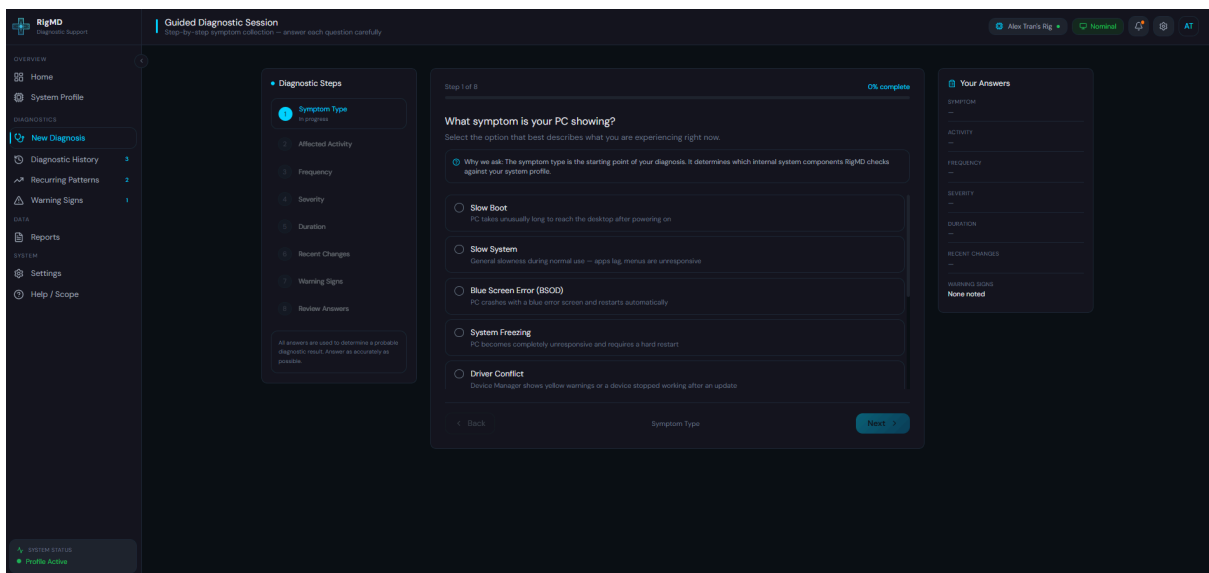
Module 2 - Structured OS Symptom Intake Module

This module guides the user through controlled diagnostic questions and also supports automatic diagnosis from fresh telemetry evidence. It collects consistent Windows-observable symptom data without requiring the user to determine which technical details are relevant. Automatic diagnosis is evidence-based: the selected scenario alone is not enough to produce a diagnosis when the required telemetry does not support it.

2.1 Guided Diagnostic Intake

User Interface Design

The Guided Diagnostic Intake screen presents a step-by-step questionnaire for one diagnostic session. The layout includes a diagnostic step list, progress indicator, current question card, selectable answer choices, navigation controls, and a running summary of the user's answers. The screen supports structured collection of symptom type, affected activity, frequency, severity, duration, recent changes, warning signs, and final review. The automatic diagnosis flow can create an Agent scan command, wait for the Agent to claim and complete it, retrieve the correlated snapshot, and then generate a result from fresh evidence.



Front-end Components for Guided Diagnostic Intake

Component Name	Description and Purpose	Component Type / Format
NewDiagnosisView	Displays guided diagnosis, automatic diagnosis modes, Agent scan command flow, and evidence preview.	React Page

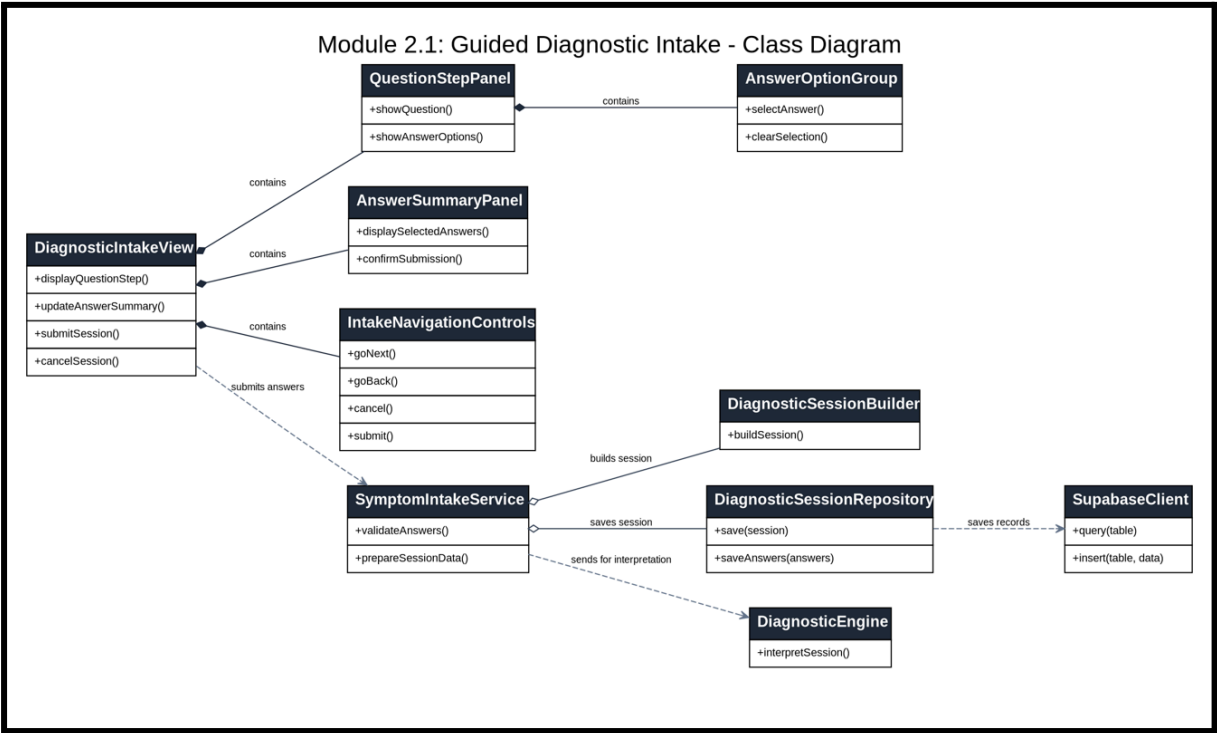
DiagnosisMode Controls	Allows the user to choose guided or automatic diagnosis behavior.	React Controls
AnswerOptionGroup	Displays selectable answer options for each guided question.	React Control Group
Agent Scan Status UI	Shows scan command creation, polling, completion, and snapshot retrieval state.	React UI State
IntakeNavigationControls	Provides Back, Next, Cancel, Submit, and scan-related controls.	React Button Group

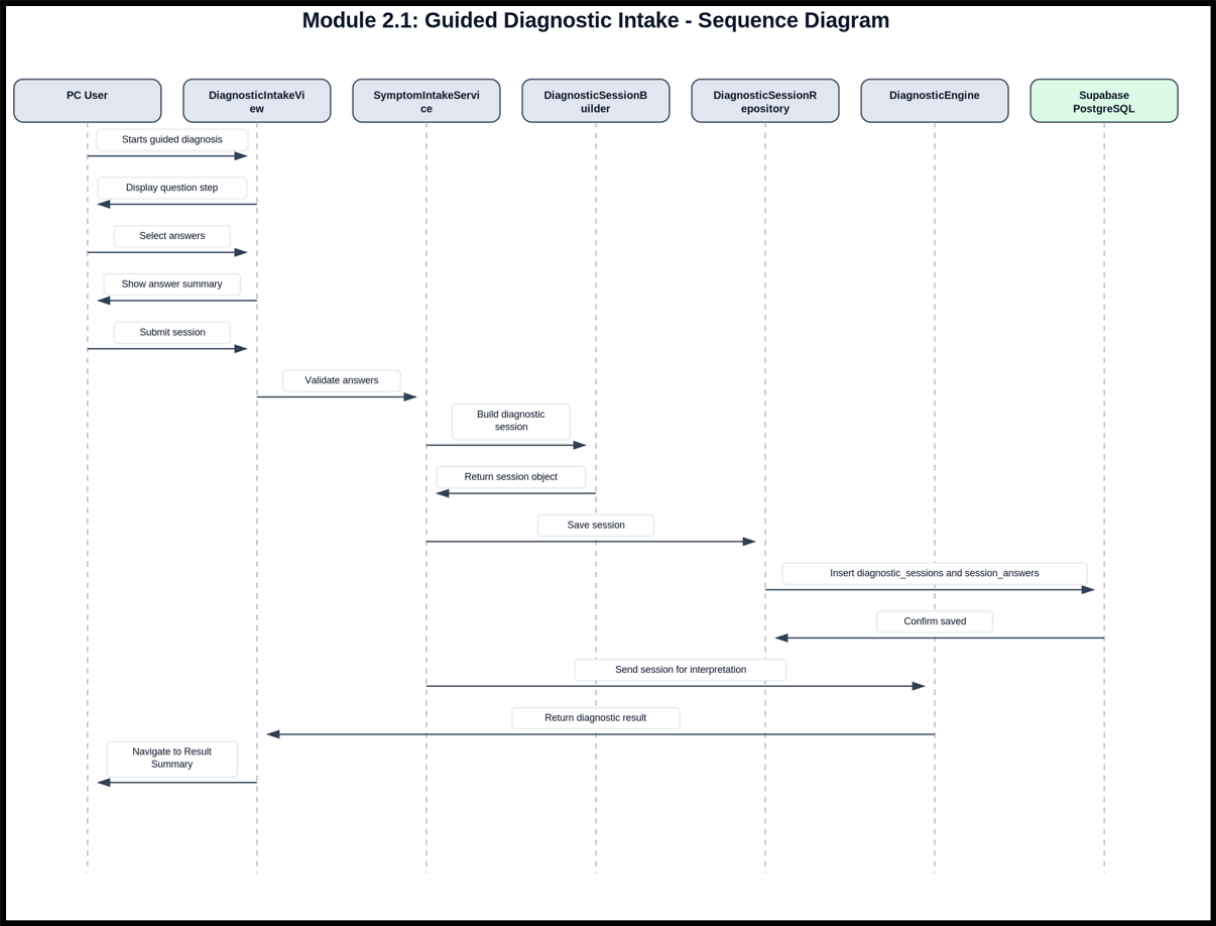
Back-end Components for Guided Diagnostic Intake

Component Name	Description and Purpose	Component Type / Format
DiagnosticEngineService	Validates intake, gathers telemetry, runs deterministic rules, gets explanation, and persists output.	C# Application Service
AutomaticDiagnosisService	Builds evidence-based automatic diagnosis from fresh scope-aware telemetry and Agent snapshots.	C# Application Service
AgentController / AgentRepository	Creates scan_system_profile commands, tracks command state, and retrieves correlated snapshots.	ASP.NET Core Controller + Npgsql Repository

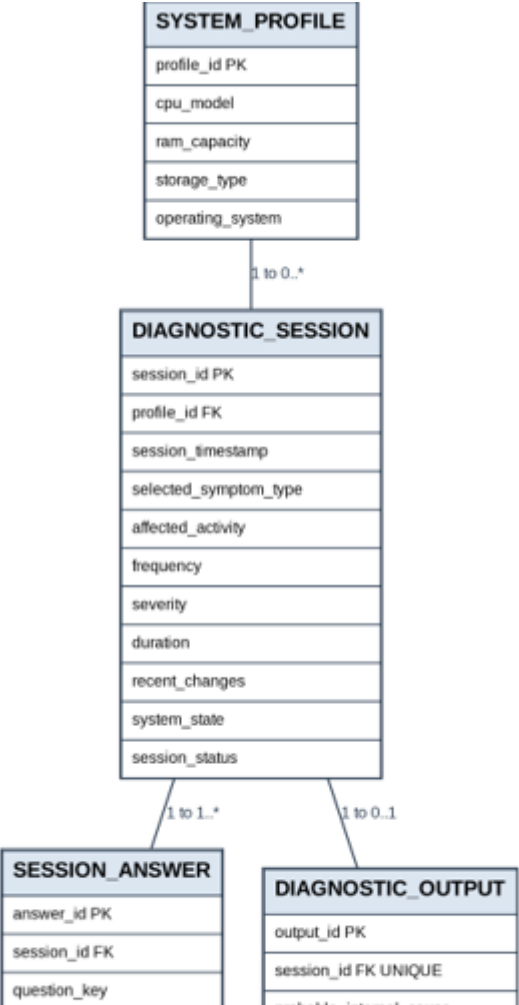
DiagnosticEngine	Receives structured intake and telemetry for deterministic interpretation.	C# Domain Rule Engine
------------------	--	-----------------------

Object-Oriented Components





Data Design / ERD or Schema for Guided Diagnostic Intake



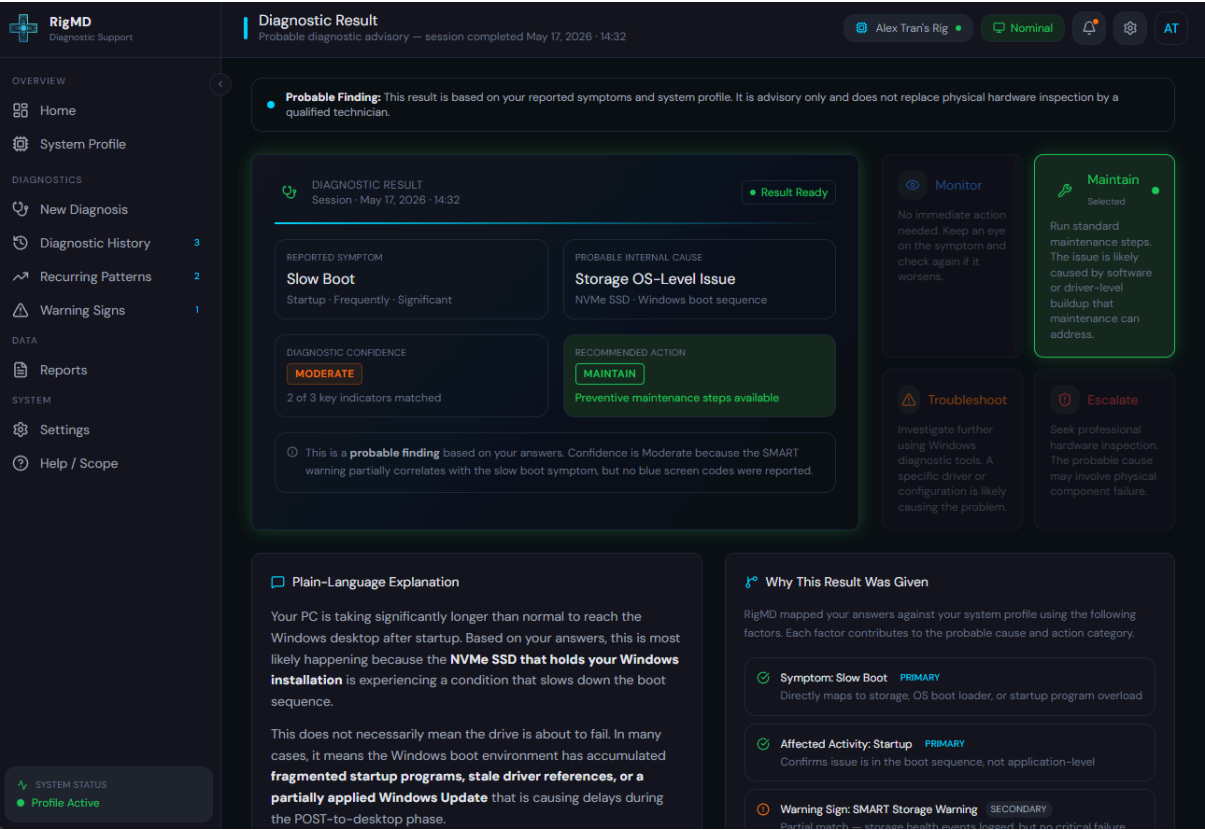
Module 3 - Internal Diagnostic Interpretation Engine

This module represents the result interpretation stage after guided or automatic diagnosis is submitted. The diagnostic result is shown as a scrollable interface with result summary, supporting evidence, recommended next steps, decision confidence, remediation availability where policy allows, and proof from Agent-backed commands when remediation is executed.

3.1 Diagnostic Result Summary

User Interface Design

The Diagnostic Result Summary section appears at the top of the completed diagnostic result screen. It presents the advisory notice, reported symptom, probable internal cause, diagnostic confidence, recommended action category, and selected action card. It also introduces the plain-language explanation and reasoning section so the user can quickly understand what the system found and how certain the result is.



Front-end Components for Diagnostic Result Summary

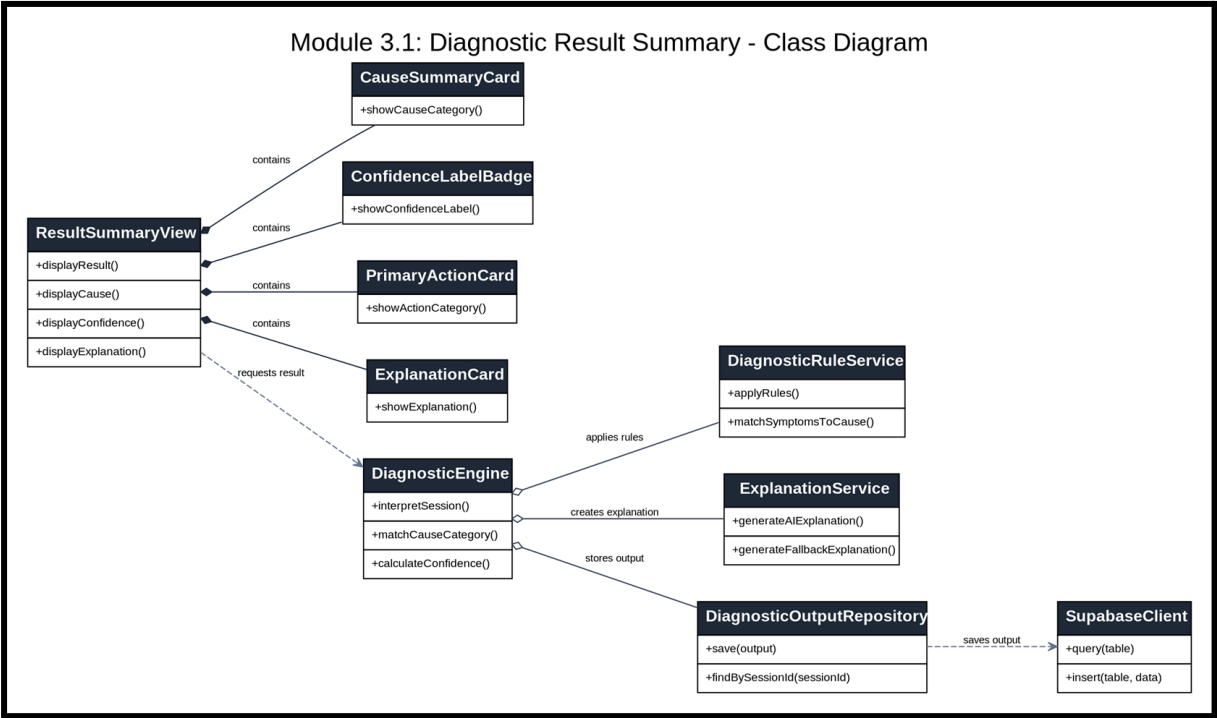
Component Name	Description and Purpose	Component Type / Format
DiagnosticResultView	Displays the probable internal cause, confidence label, evidence, and primary recommended action.	React Page
CauseSummaryCard	Shows the most likely internal cause category including Network issue only when telemetry supports it.	React Component
ConfidenceLabelBadge	Displays the diagnostic confidence level.	React Badge

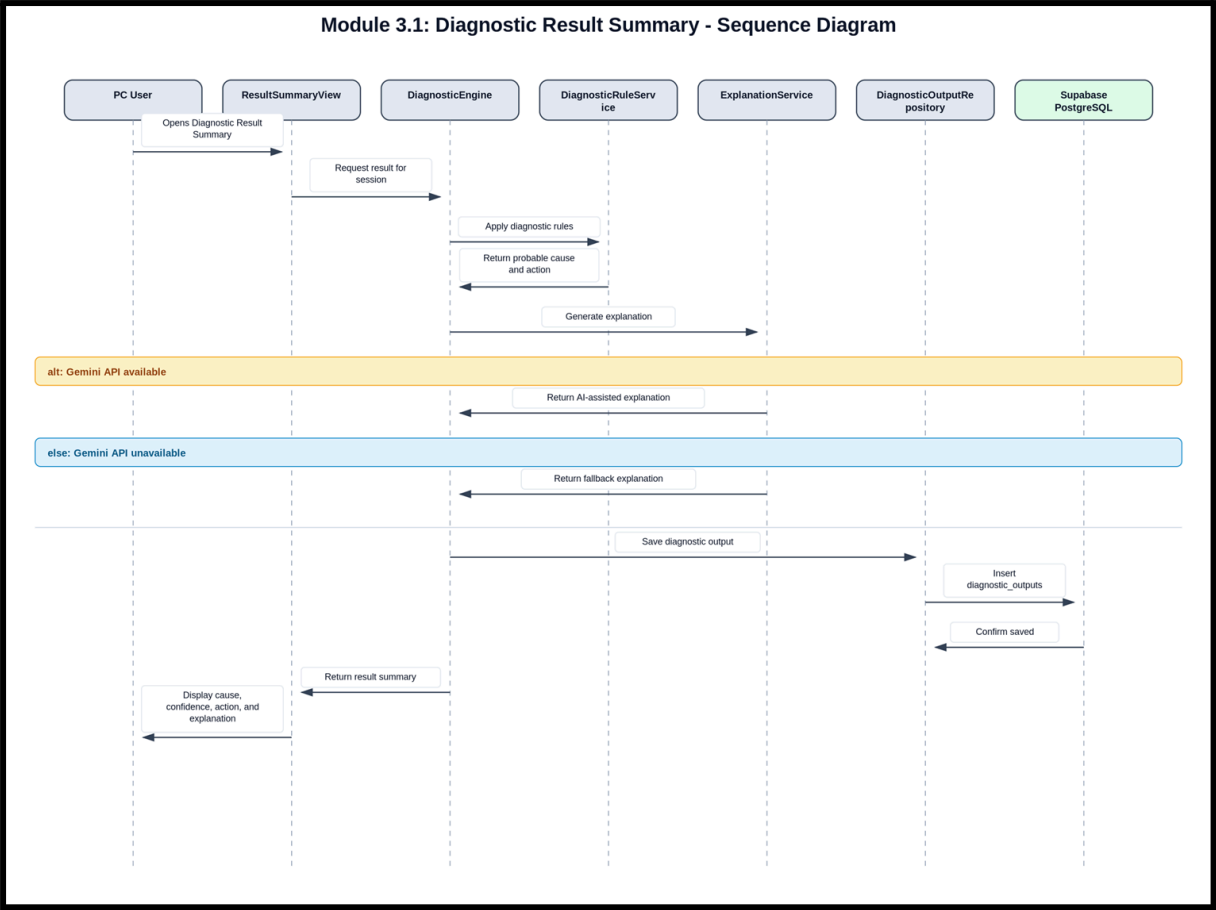
PrimaryActionCard	Displays Monitor, Maintain, Troubleshoot, Remediate, Escalate, or No Active Issue action outcomes.	React Component
ExplanationCard	Shows constrained Gemini/offline explanation text for the deterministic result.	React Component

Back-end Components for Diagnostic Result Summary

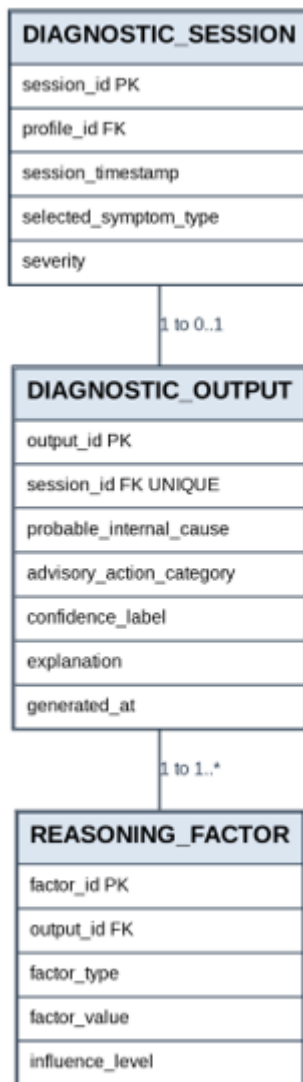
Component Name	Description and Purpose	Component Type / Format
DiagnosticEngine	Maps symptom answers, system profile data, telemetry, and history into a probable cause category.	C# Domain Rule Engine
AutomaticDiagnosisService. DiagnoseNetwork	Returns Network issue only when active adapter, IPv4, gateway, and DNS servers exist but DNS resolution fails; otherwise no network diagnosis is invented.	C# Application Rule
GeminiAiExplainer / OfflineAiExplainer	Generates explanation text without changing diagnosis, action selection, or remediation authority.	C# Infrastructure Services
DiagnosticSessionRepository	Saves diagnostic outputs, session answers, evidence, and session relationships to SQLite.	EF Core Repository

Object-Oriented Components





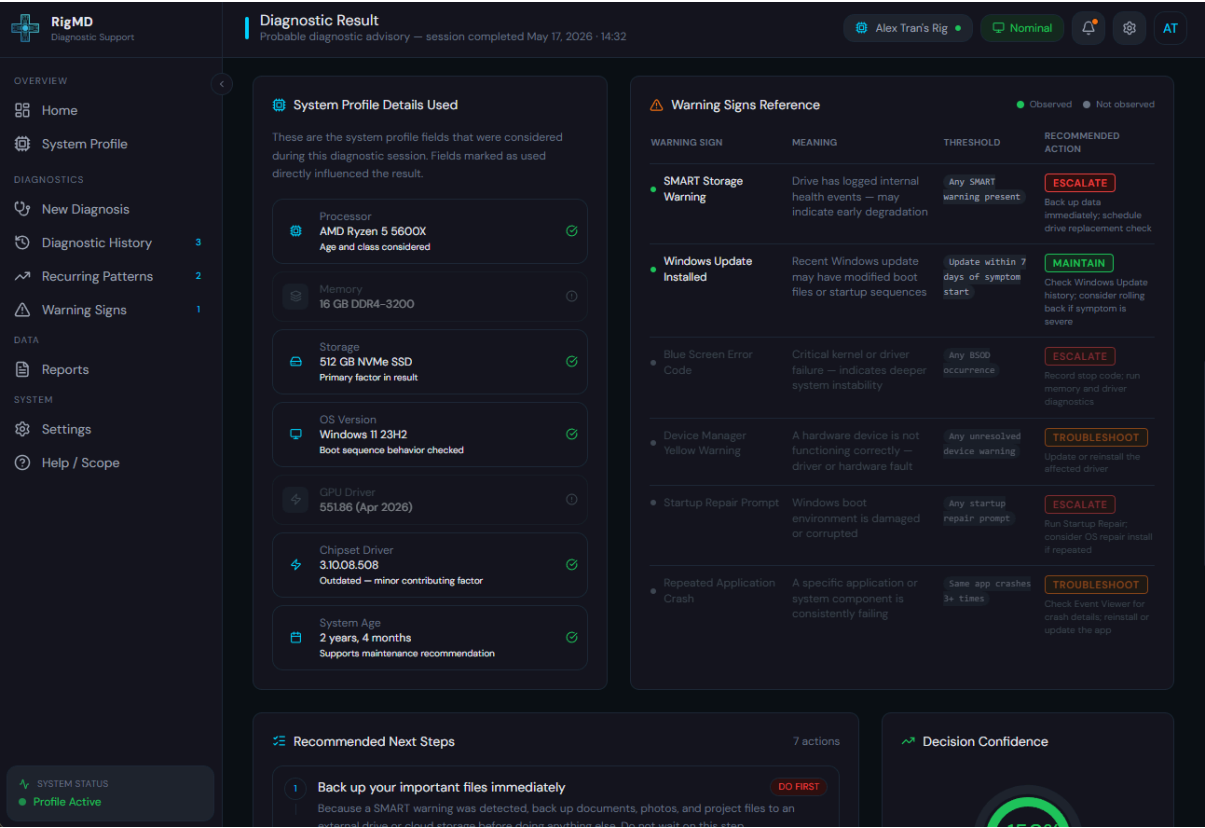
Data Design / ERD or Schema for Diagnostic Result Summary



3.2 Supporting Diagnostic Details

User Interface Design

The Supporting Diagnostic Details section explains the evidence behind the result. It shows the system profile fields considered during the session, such as processor, memory, storage, operating system version, GPU driver, chipset driver, and system age. It also displays the warning signs reference table, including warning sign, meaning, threshold, and recommended action.



Front-end Components for Supporting Diagnostic Details

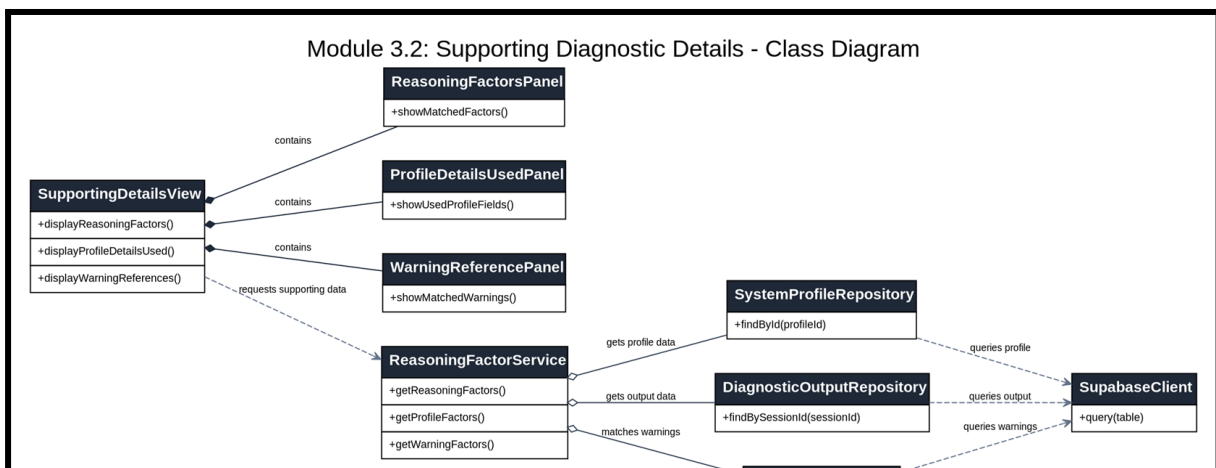
Component Name	Description and Purpose	Component Type / Format
DiagnosticSessionDetailView	Displays supporting details, Agent context, evidence, command state, and remediation proof.	React Page
ReasoningFactorsPanel	Shows the exact profile fields, symptoms, telemetry, network facts, and rule factors that influenced the result.	React Component
ProfileDetailsUsedPanel	Displays system profile and Agent snapshot values used by the diagnostic engine.	React Component

WarningReferencePanel	Shows matched warning signs, meanings, thresholds, and action categories.	React Component
-----------------------	---	-----------------

Back-end Components for Supporting Diagnostic Details

Component Name	Description and Purpose	Component Type / Format
DiagnosticOutput	Stores category, confidence, explanation, reasoning factors, warning signs, and remediation runs.	C# Domain Entity
HardwareProfileDto / NetworkStatsDto	Carries CPU, GPU, RAM, storage, process, and network evidence used by diagnosis.	C# DTO Models
DiagnosticSessionRepository	Retrieves saved diagnostic output details from SQLite.	EF Core Repository
AgentSnapshotRecord	Represents the Supabase-stored hardware JSON snapshot correlated to an Agent command.	Agent Persistence Record

Object-Oriented Components



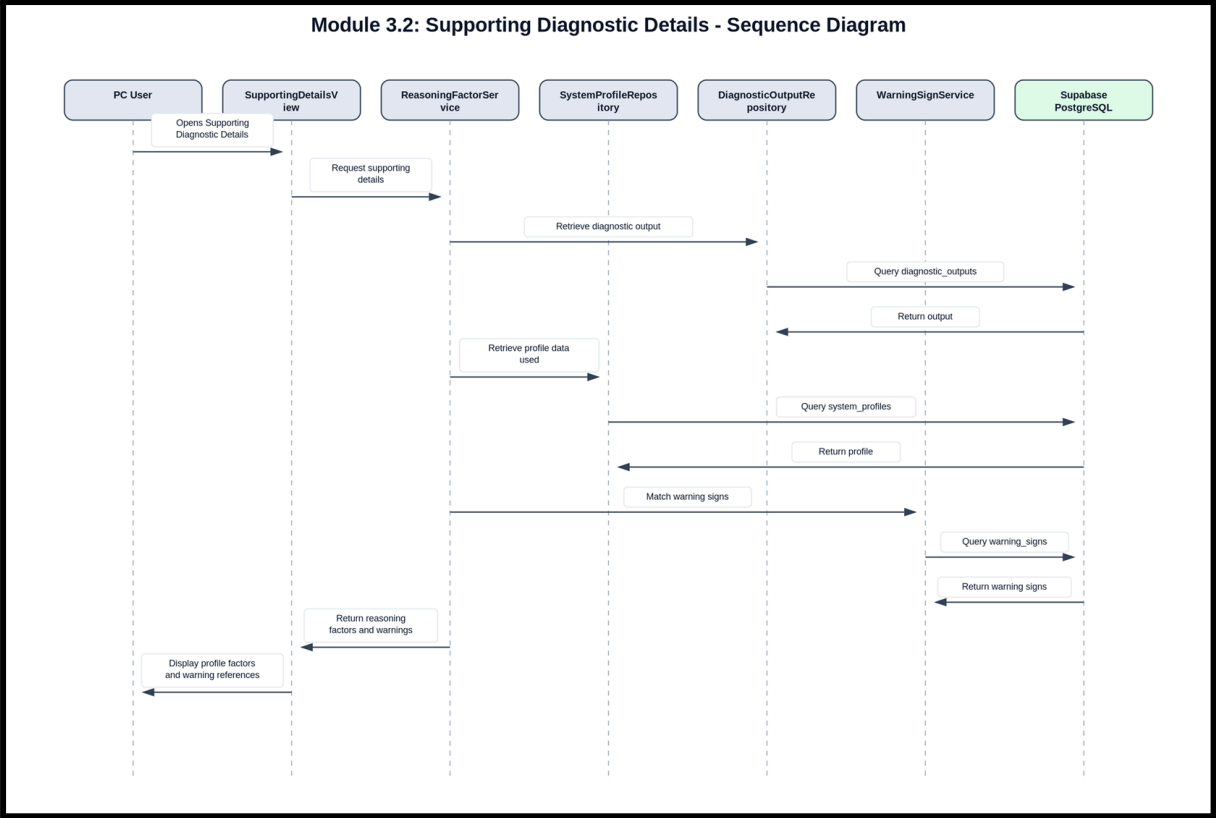
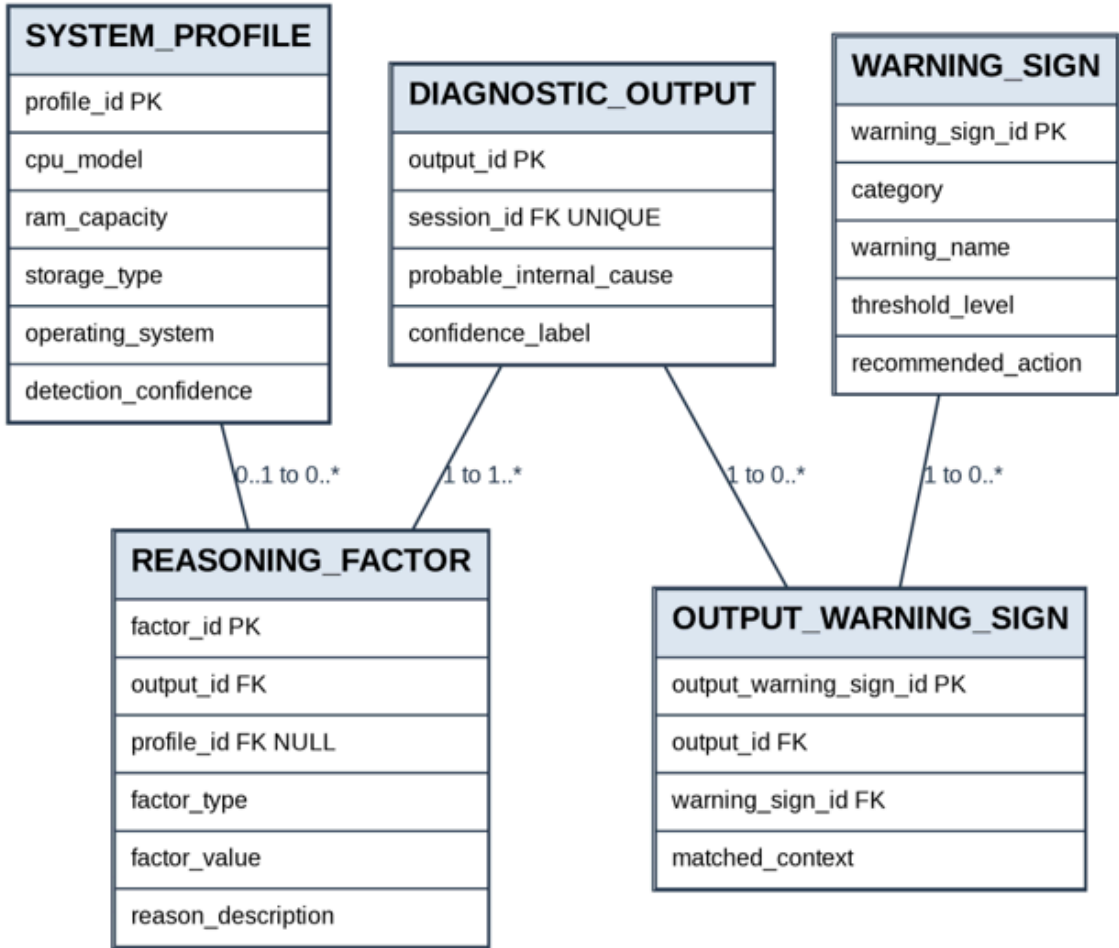


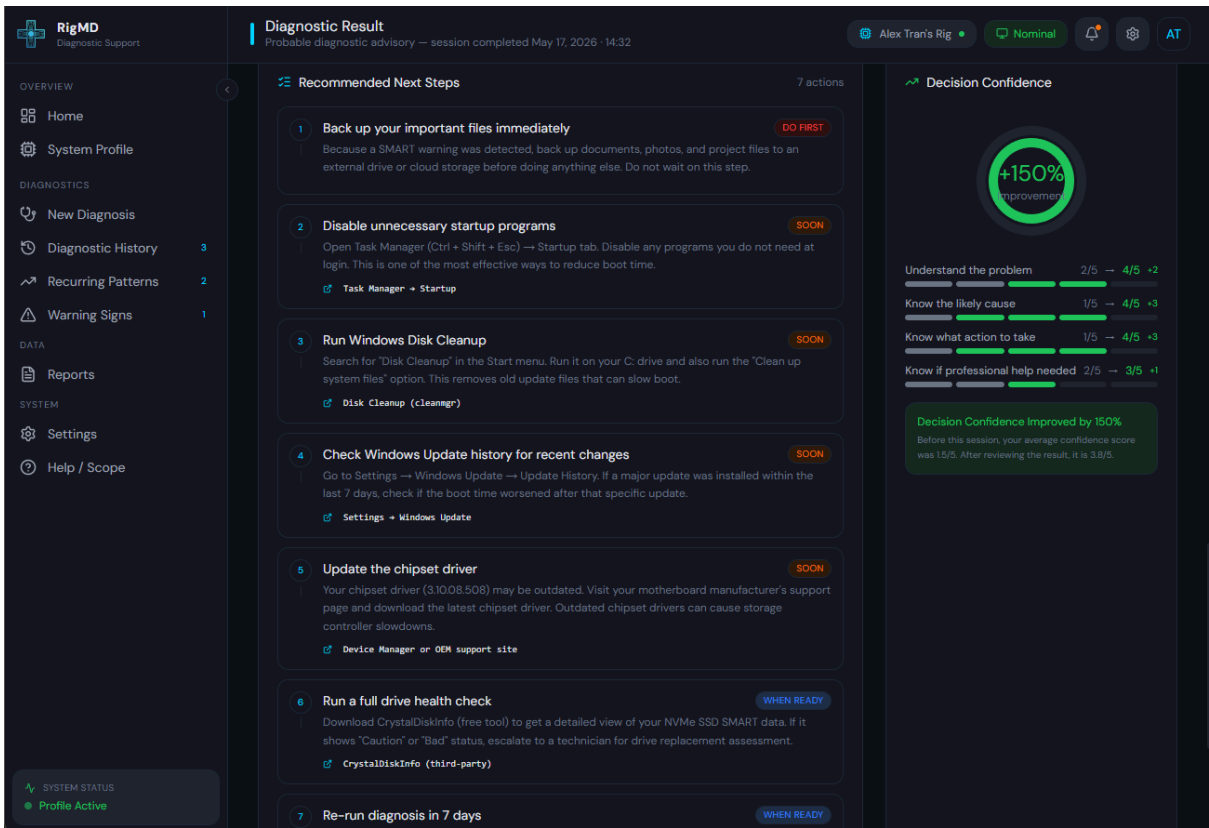
Table 3.3.2C. Data Design / ERD or Schema for Supporting Diagnostic Detail



3.3 Recommended Actions and Decision Confidence

User Interface Design

The Recommended Actions and Decision Confidence section shows the lower part of the diagnostic result screen. It lists the user's next steps in priority order and uses labels such as Do First, Soon, and When Ready. It also includes a decision confidence panel showing how the session helps the user understand the problem, identify the likely cause, choose an action, and decide whether professional help is needed.



Front-end Components for Recommended Actions and Decision Confidence

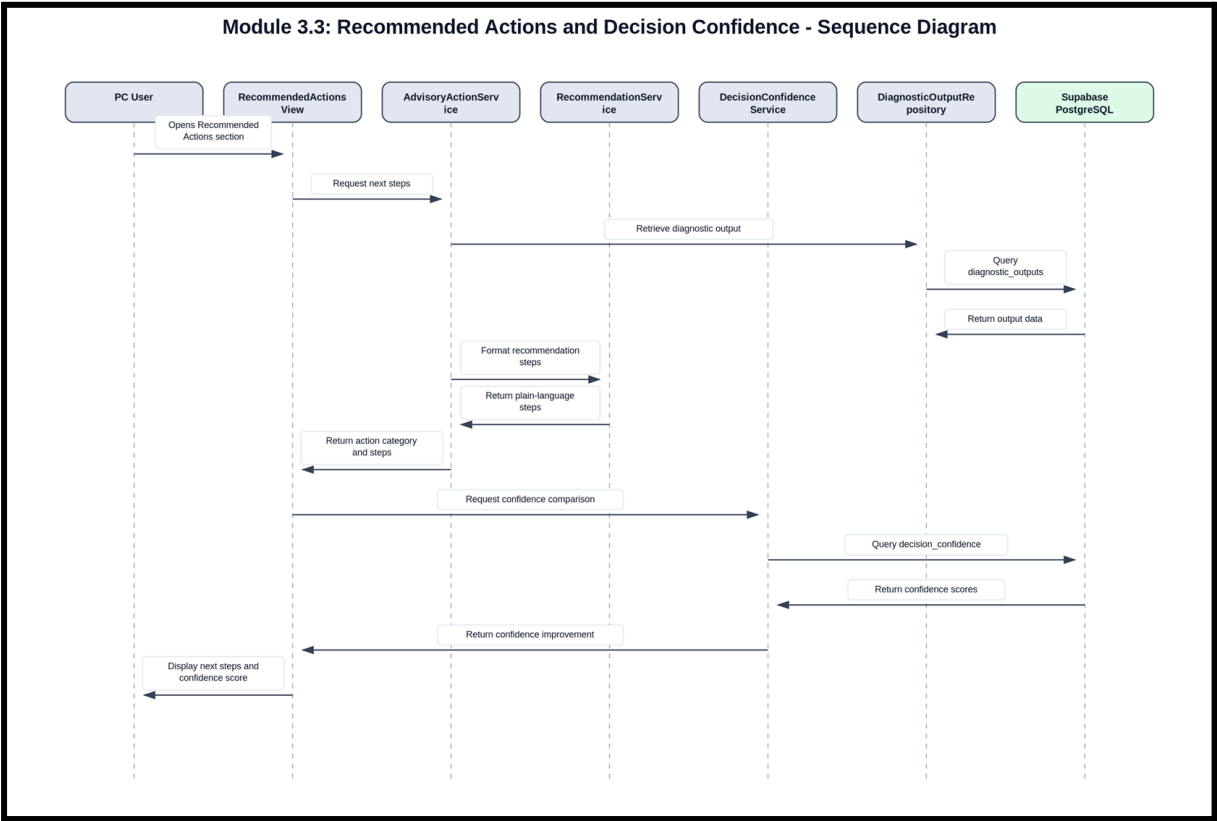
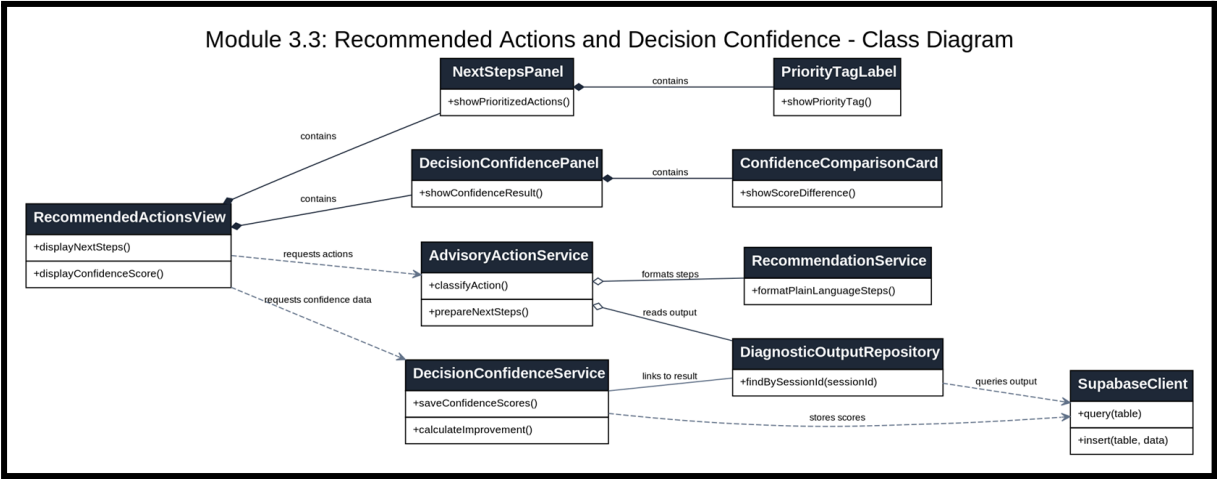
Component Name	Description and Purpose	Component Type / Format
AutonomyRemediationPanel	Displays available remediation plans, Agent-backed actions, command polling, proof, and safety decisions.	React Component

NextStepsPanel	Shows prioritized user actions and remediation guidance.	React Component
PriorityTagLabel	Displays labels such as DO FIRST, OBSERVE, VERIFY, or ESCALATE.	React Badge
DecisionConfidencePanel	Displays the user's decision confidence and resolution state.	React Component
ProofList / ResultCard	Displays execution proof and status returned by Agent command completion.	React Components

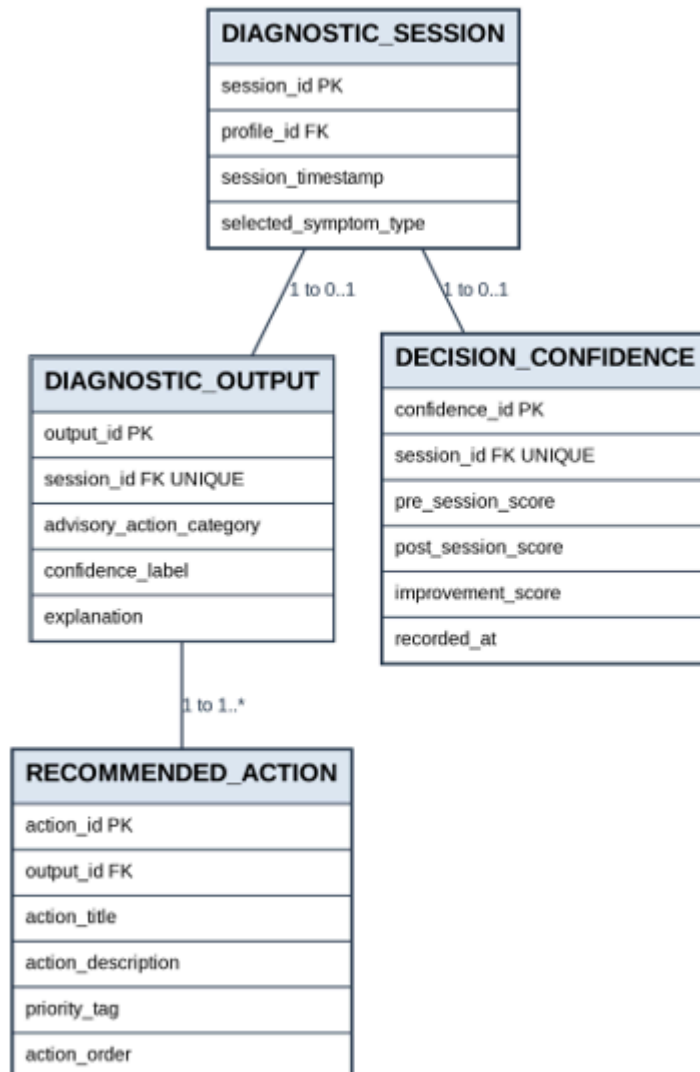
Back-end Components for Recommended Actions and Decision Confidence

Component Name	Description and Purpose	Component Type / Format
RemediationRegistry	Defines fixed allowlisted actions including clear_user_temp_files, restart_explorer, and flush_dns.	C# Application Service
AgentController	Queues confirmed Agent remediation commands for clear_user_temp_files and flush_dns.	ASP.NET Core Controller
ClearTempFilesAction / FlushDnsAction	Implement actual temp cleanup and DNS cache flush behavior with output/proof.	C# Infrastructure Action Classes
Agent Worker	Claims pending commands, executes only supported command types, and completes/fails commands with JSON result proof.	Windows Background Service

Object-Oriented Components



Data Design / ERD or Schema for Recommended Actions and Decision Confidence



Module 4 - Advisory Action Module

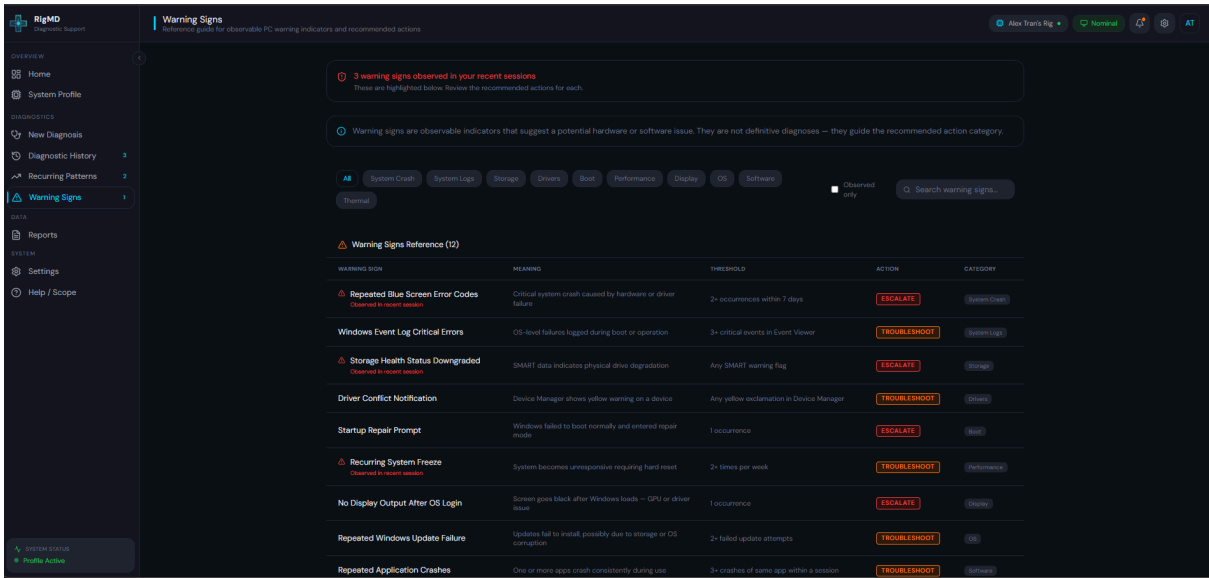
This module supports the system's action-classified advisory output. It provides the warning sign reference library and connects advisory results to controlled remediation planning, helping users understand when to monitor, maintain, troubleshoot, remediate with approval, or escalate

the issue for professional inspection. Network issue output is only produced when network telemetry proves a DNS resolution failure despite an active adapter, IPv4 address, default gateway, and DNS servers.

4.1 Warning Signs Reference

User Interface Design

The Warning Signs Reference screen provides a standalone guide to observable PC warning indicators. It displays active warning counts, an explanation note, category filters, an observed-only toggle, search field, and a table of warning signs. Each warning sign entry should include its meaning, threshold, recommended action, and category. This helps users recognize escalation points even outside a completed diagnostic session.



Front-end Components for Warning Signs Reference

Component Name	Description and Purpose	Component Type / Format
WarningSignsView	Displays the standalone warning signs reference library.	React Page

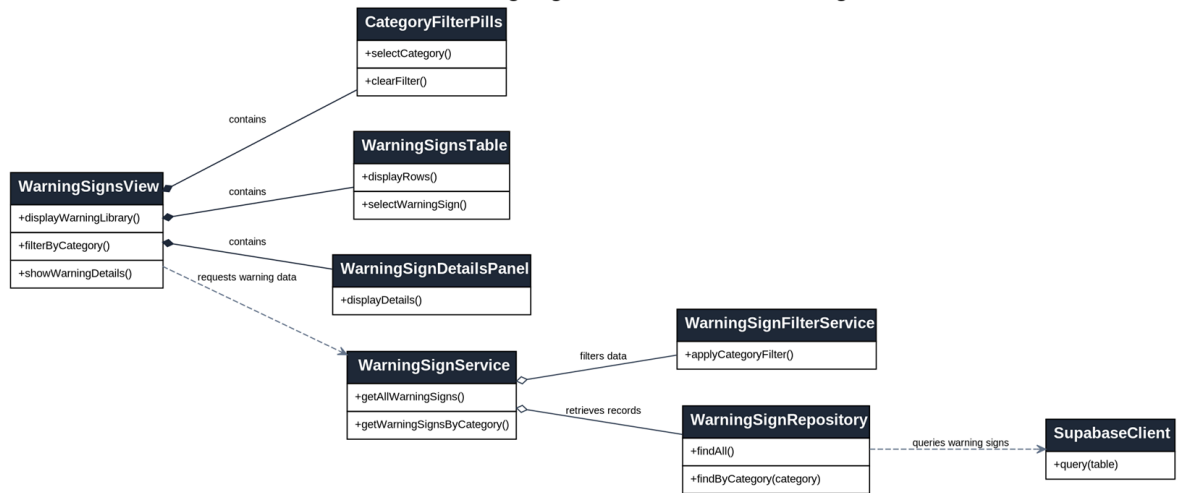
CategoryButton	Allows filtering by categories such as Storage, Crash, Driver, Boot, or Performance.	React Filter Control
WarningSignsTable / Warning Rows	Shows warning sign name, meaning, threshold, occurrence count, and recommended action.	React Table
ObservedToggle	Filters the library to observed warning signs.	React Toggle

Back-end Components for Warning Signs Reference

Component Name	Description and Purpose	Component Type / Format
WarningSignService	Retrieves, normalizes, and filters warning sign reference data.	C# Application Service
WarningSignsController	Exposes GET /api/warning-signs/reference.	ASP.NET Core Controller
WarningSignFilter Logic	Applies category, text search, and observed-only filtering.	C# Application / API Logic

Object-Oriented Components

Module 4.1: Warning Signs Reference - Class Diagram



Module 4.1: Warning Signs Reference - Sequence Diagram

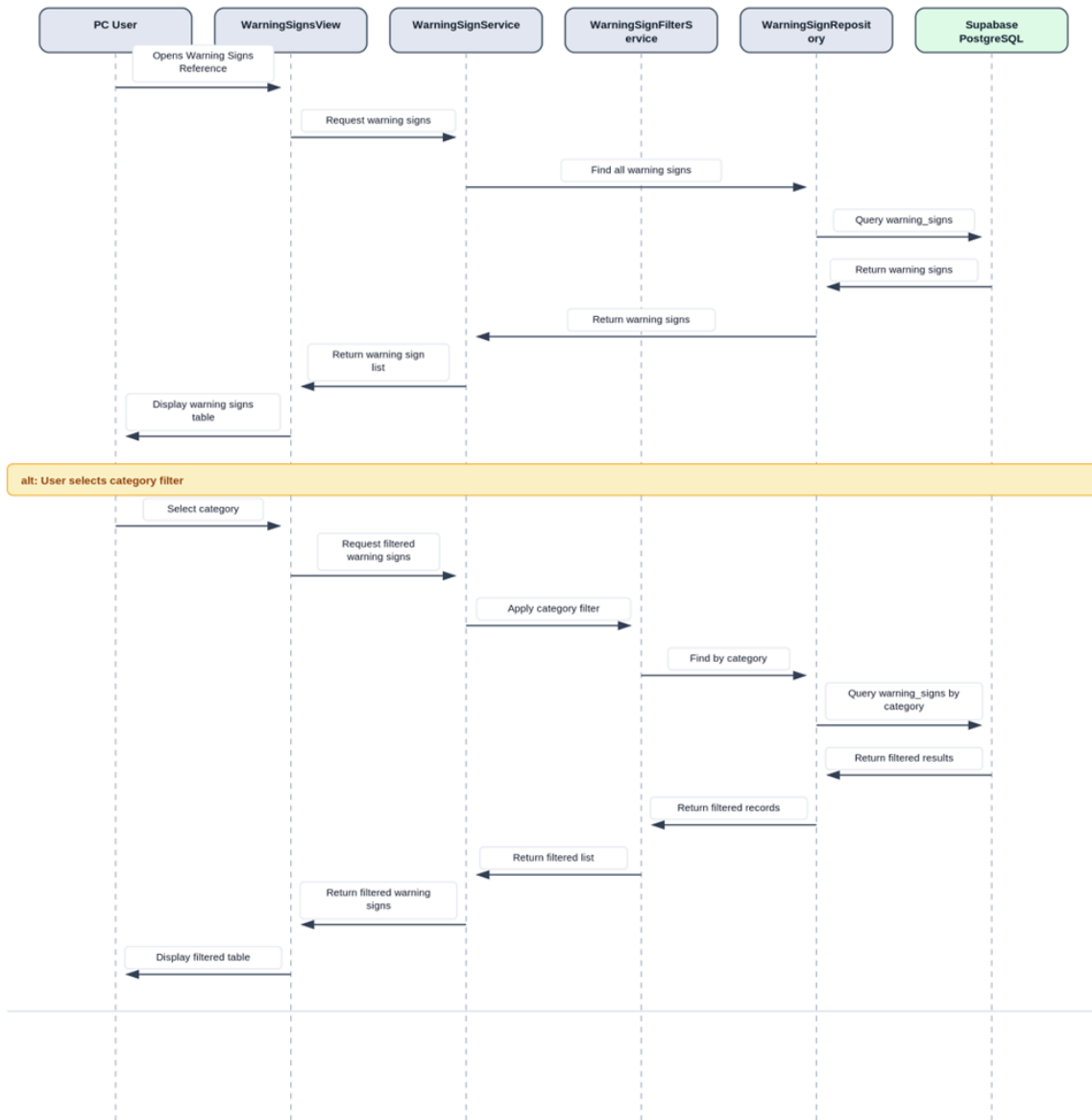
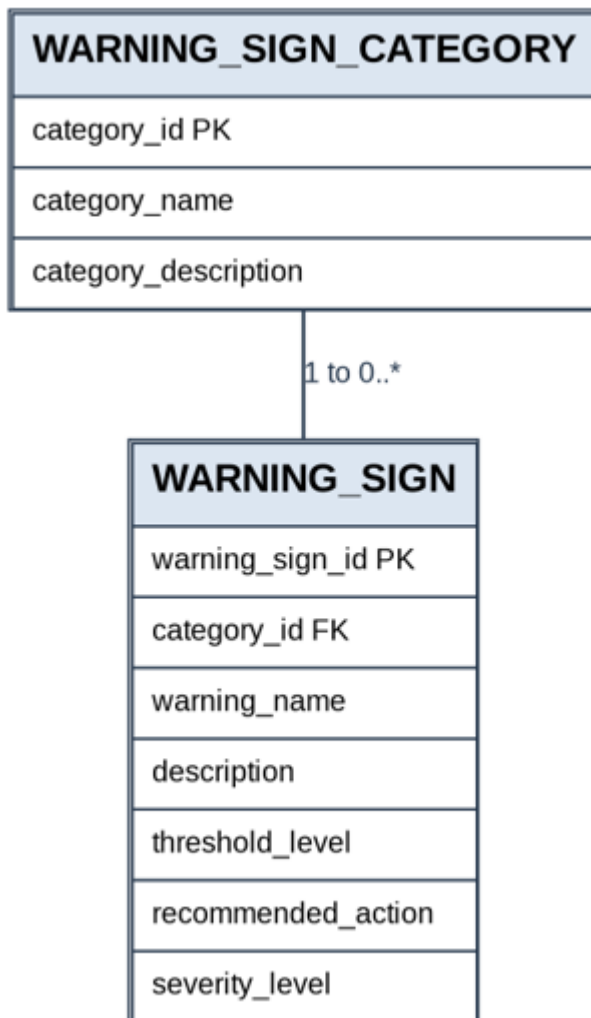


Table 3.4.1C. Data Design / ERD or Schema for Warning Signs Reference



Module 5 - Transparent Result Dashboard and Report Output Module

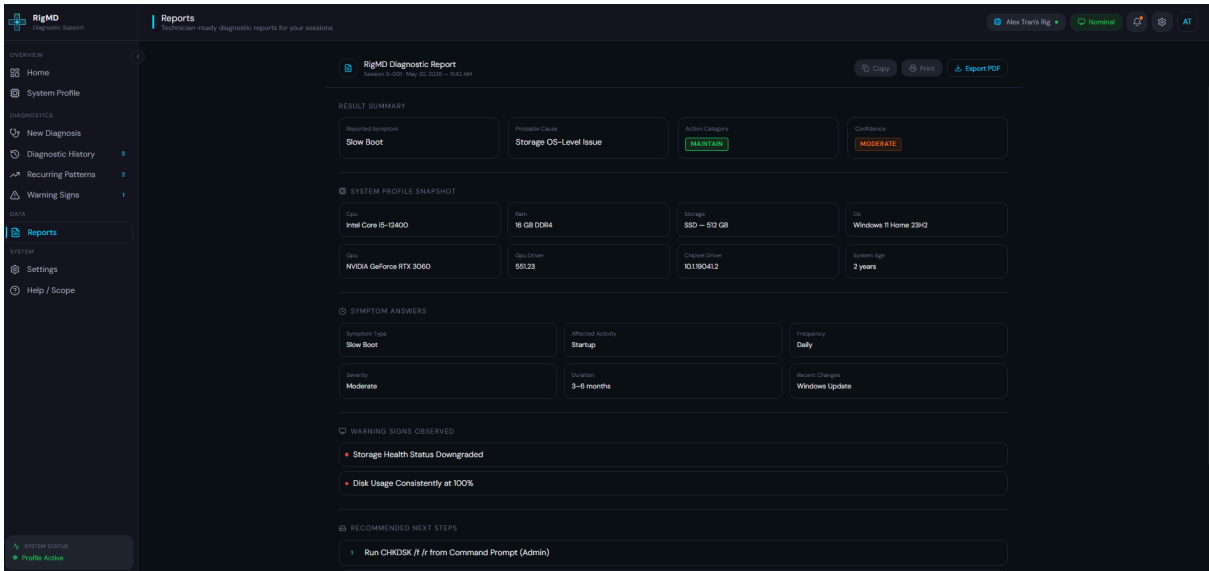
This module supports transparent result output based on saved diagnostic sessions. The latest implementation emphasizes dashboard summaries and detailed session views that organize diagnostic information, telemetry evidence, network evidence, warning signs, Agent command state, remediation proof, and resolution status for user review or technician consultation.

5.1 Reports

User Interface Design

The Reports screen allows the user to select a completed diagnostic session and view a technician-ready report. The interface should include a session selector, result summary cards,

system profile snapshot, symptom answers, warning signs observed, and action buttons for copy, print, and PDF export. The report supports user preparation before repair, troubleshooting, or professional consultation.



Front-end Components for Reports

Component Name	Description and Purpose	Component Type / Format
DiagnosticSessionDetailView	Allows the user to open a completed session and review technician-ready details.	React Page
SessionSelector / History Link	Lets the user choose which diagnostic session to inspect.	React Navigation
ReportPreviewPanel	Displays summary, profile snapshot, answers, evidence, network facts, warning signs, command state, and remediation proof.	React Detail Panel

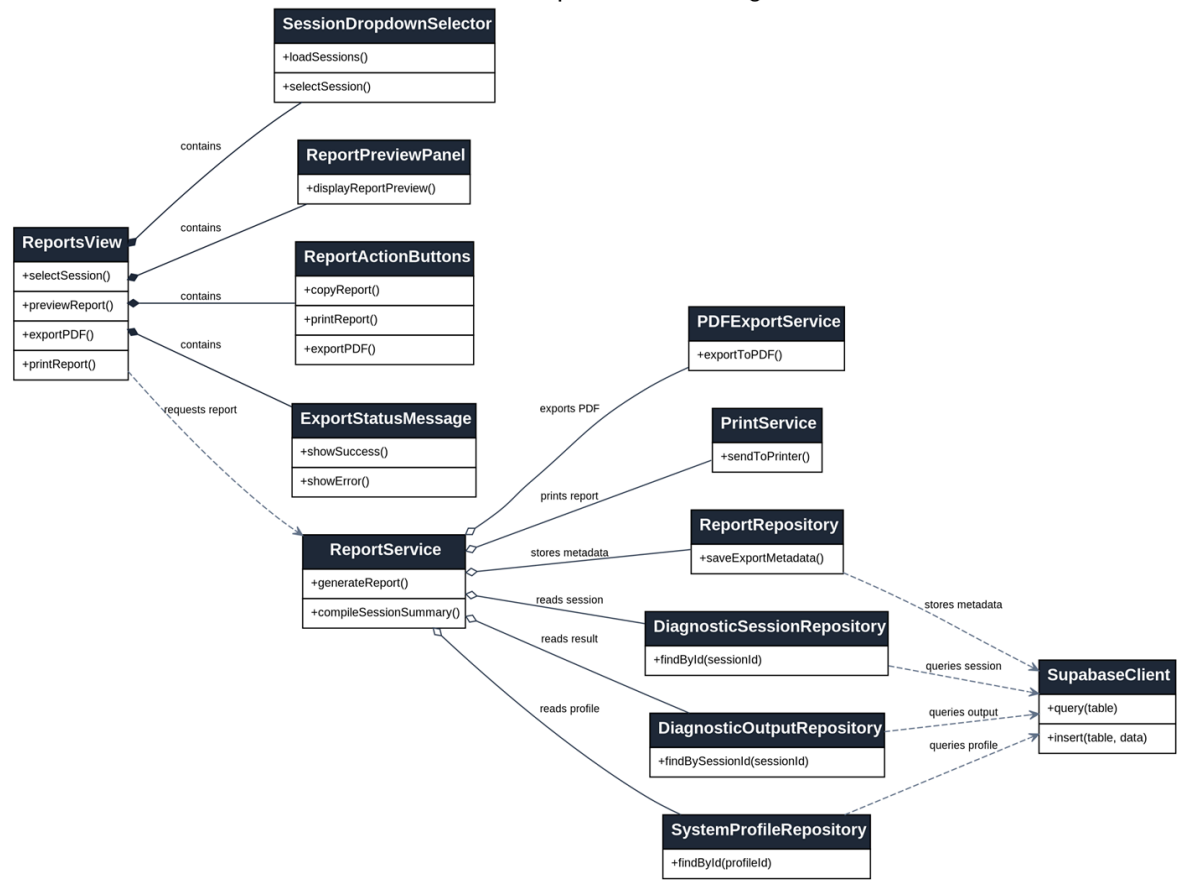
ReportActionButtons	Provides copy/print/export-oriented actions where supported by the frontend.	React Button Group
ExportStatusMessage	Shows success or error messages after report-oriented actions.	React Alert

Back-end Components for Reports

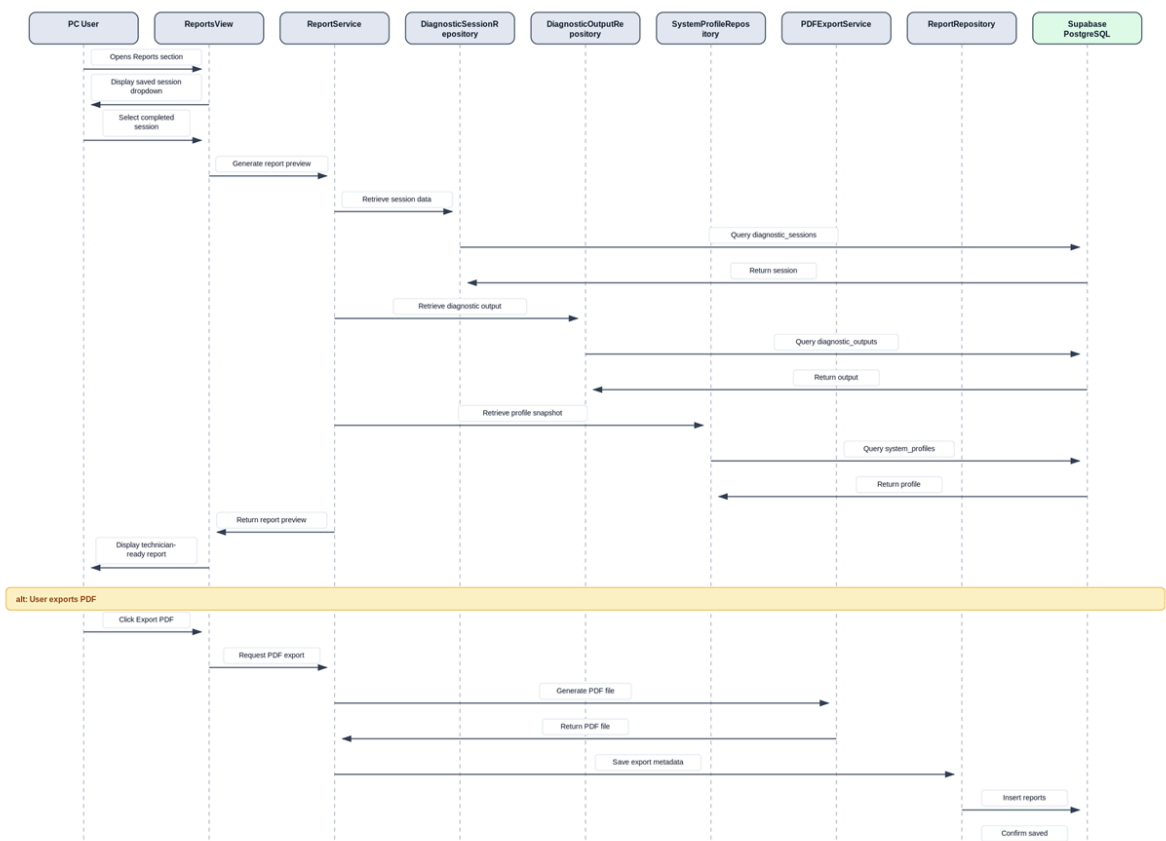
Component Name	Description and Purpose	Component Type / Format
Dashboard/Detail Services	Compile session data, profile snapshot, symptom answers, network evidence, warning signs, and results for display.	C# Application/API Services
DiagnosticSessionRepository	Retrieves selected session records from SQLite.	EF Core Repository
AgentRepository	Retrieves Agent command result JSON and latest snapshots from Supabase where needed.	Npgsql/Supabase Repository
RemediationRepository	Provides remediation run and action attempt history for the session.	EF Core Repository
Future PDFExportService	Converts a report view into a PDF file when report export is finalized.	Planned Utility Service
Future PrintService	Sends the generated report to the system print function when implemented.	Planned Utility Service

Object-Oriented Components

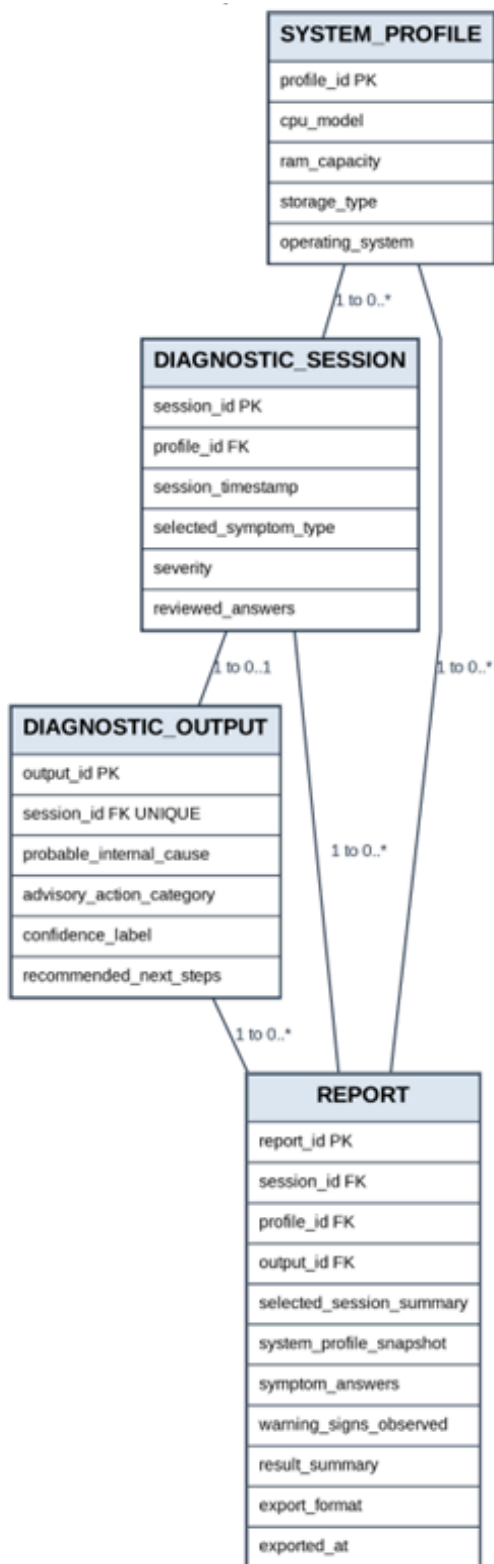
Module 5.1: Reports - Class Diagram



Module 5.1: Reports - Sequence Diagram



Data Design / ERD or Schema for Reports



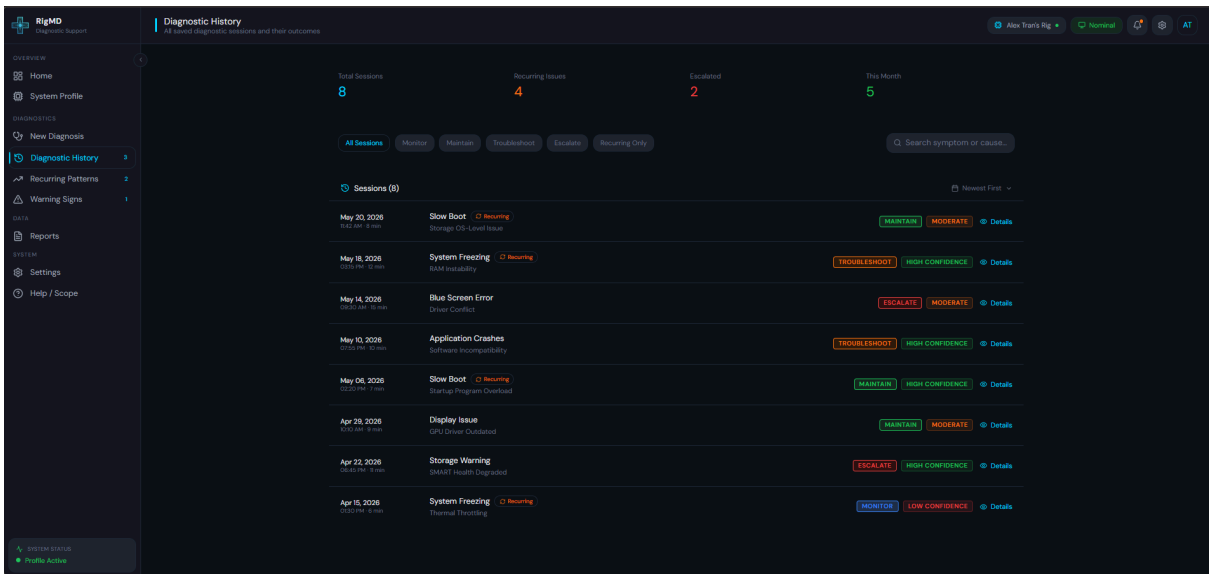
Module 6 - Session History and Recurring Pattern Detection Module

This module provides the longitudinal tracking layer of RigMD. It stores completed diagnostic sessions and remediation attempts locally in SQLite, detects repeated symptoms or internal cause categories across sessions, and uses failed action history to deprioritize weaker future remediation candidates. Agent devices, snapshots, and command state are stored through Supabase-backed AgentRepository records so the API and Windows Agent can coordinate across processes.

6.1 Diagnostic History

User Interface Design

The Diagnostic History screen lists saved diagnostic sessions in chronological order. It includes summary metrics, action category filters, search, sorting, session entries, symptom titles, probable causes, action categories, confidence labels, recurring tags, and detail links. The screen helps users review prior results and retrieve previous advisory outputs when needed.



Front-end Components for Diagnostic History

Component Name	Description and Purpose	Component Type / Format
----------------	-------------------------	-------------------------

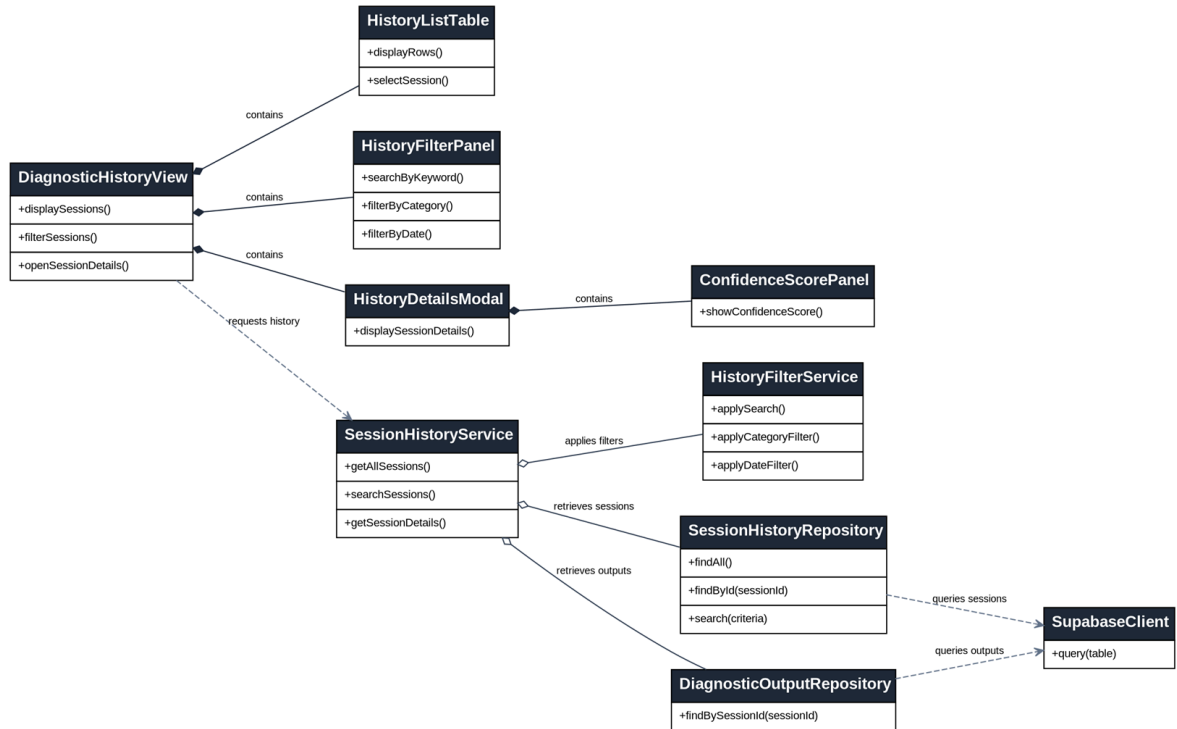
DiagnosticHistoryView	Displays saved diagnostic sessions in chronological order.	React Page
HistoryListTable / SessionRow	Shows previous sessions, dates, symptoms, probable causes, action outcomes, and remediation status.	React Table
HistoryFilterPanel / FilterButton	Allows searching and filtering by symptom, date, cause, status, or action category.	React Filter Panel
SessionDetailLink	Opens the full details of a selected past session.	React Navigation
MetricCard	Shows session totals and confidence/history metrics.	React Component

Back-end Components for Diagnostic History

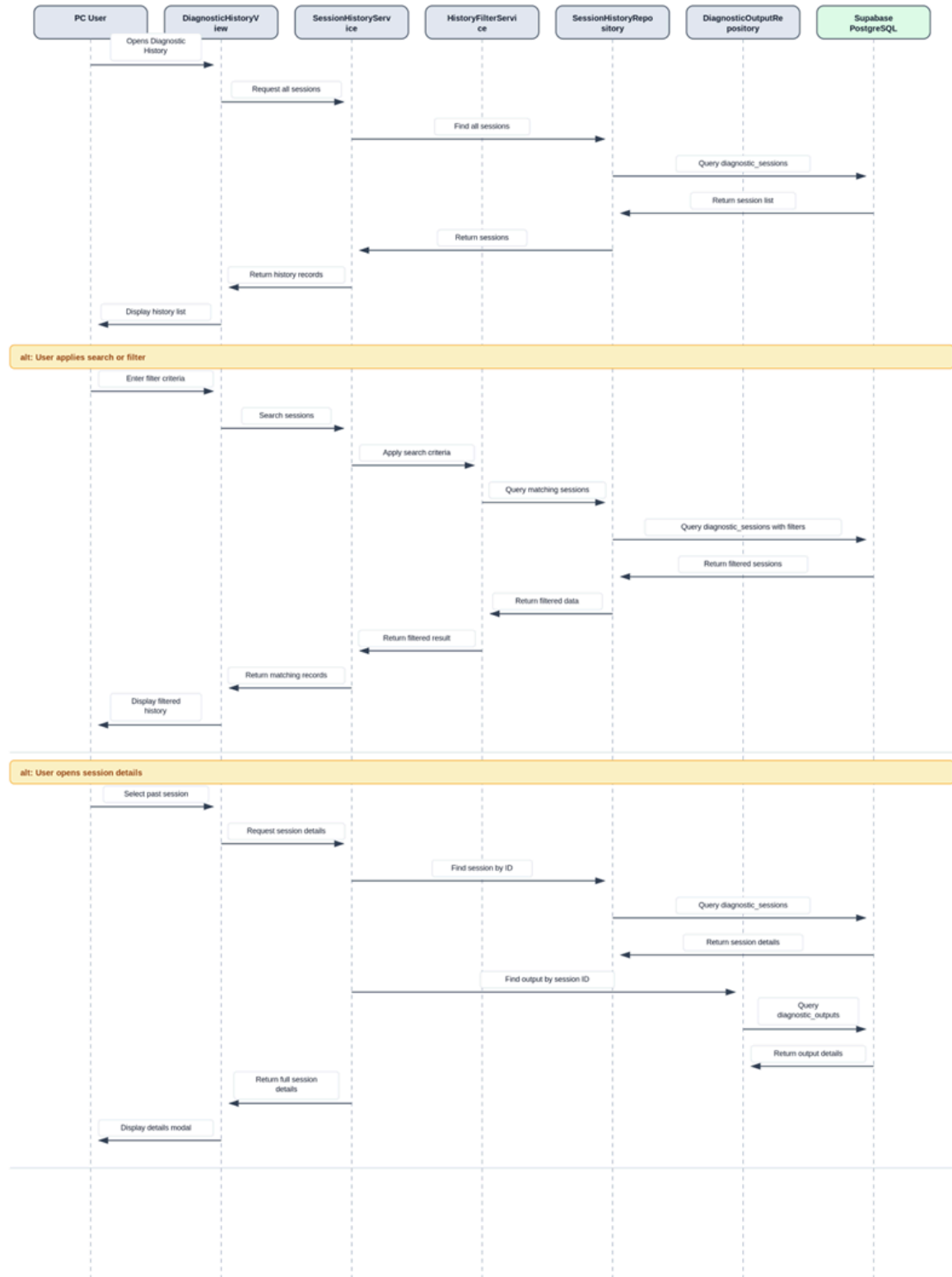
Component Name	Description and Purpose	Component Type / Format
DiagnosticSessionRepository	Retrieves, searches, and filters saved diagnostic sessions.	EF Core Repository
DiagnosticSession	Represents a persisted diagnosis session.	C# Domain Entity
DiagnosticOutput	Provides result data connected to each session.	C# Domain Entity
RecurringPatternService	Applies search, recurrence, and trend logic to session history.	C# Application Service

Object-Oriented Components

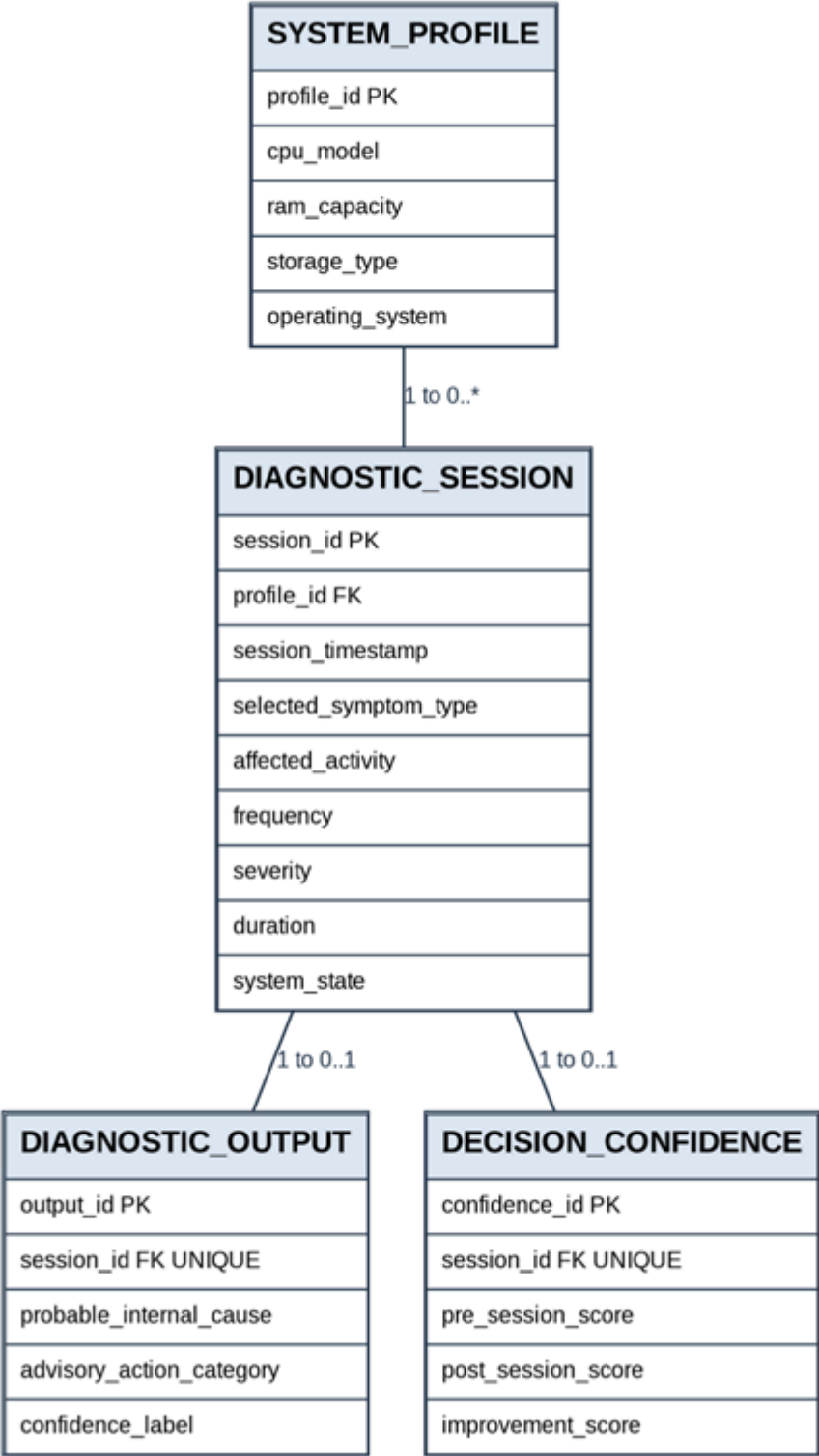
Module 6.1: Diagnostic History - Class Diagram



Module 6.1: Diagnostic History - Sequence Diagram



Data Design / ERD or Schema for Diagnostic History



6.2 Recurring Patterns

User Interface Design

The Recurring Patterns screen shows repeated symptoms and internal cause categories detected across saved diagnostic sessions. The interface should display recurring issue count, worsening trend count, action escalation count, total occurrences, repeated symptom cards, first seen date, latest date, trend label, and previous-to-current action comparison. This screen helps users identify whether an issue is stable, improving, worsening, or escalating.

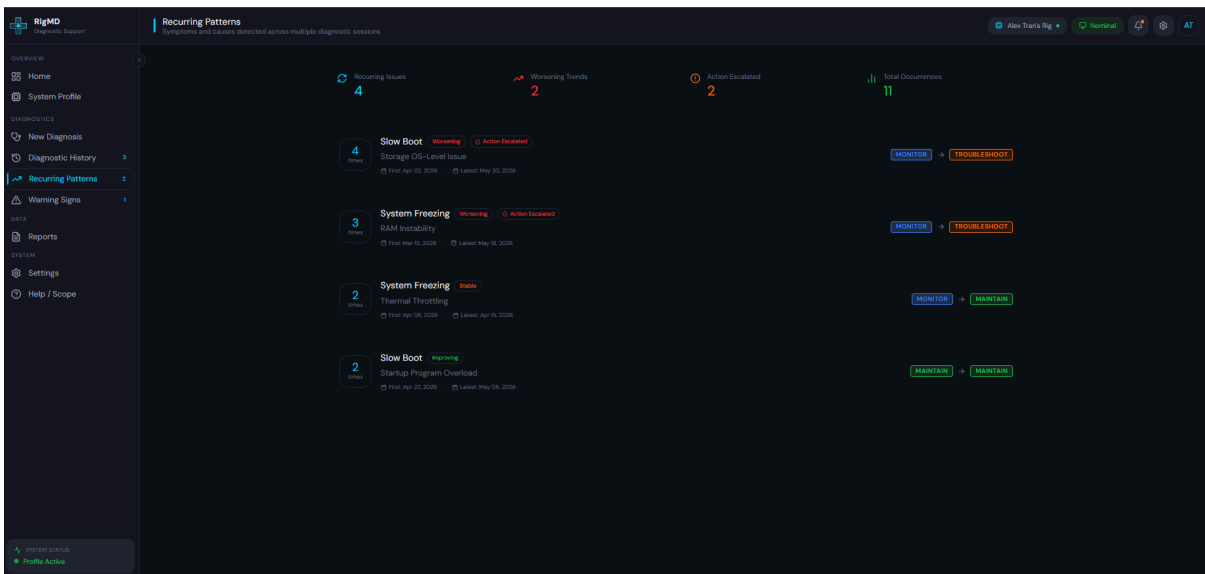


Table 3.6.2A. Front-end Components for Recurring Patterns

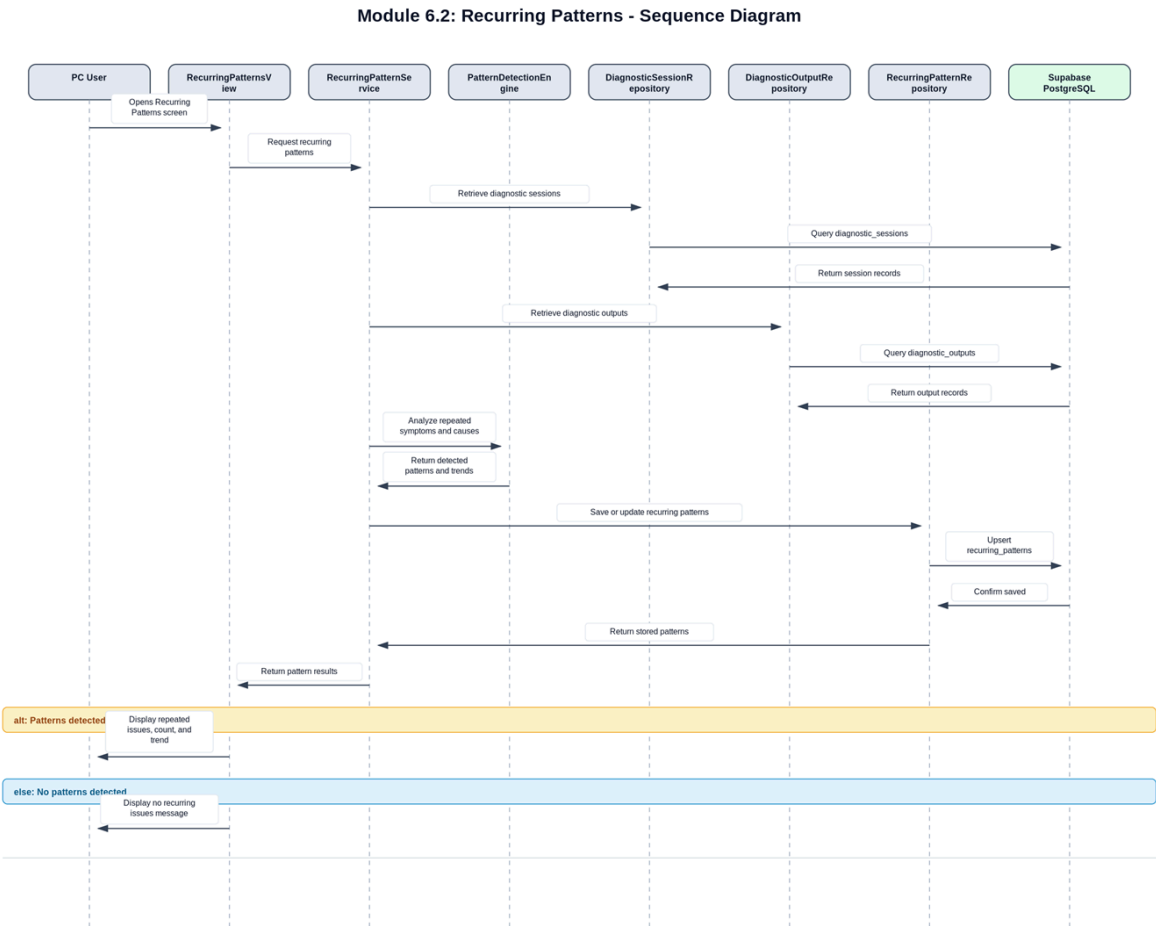
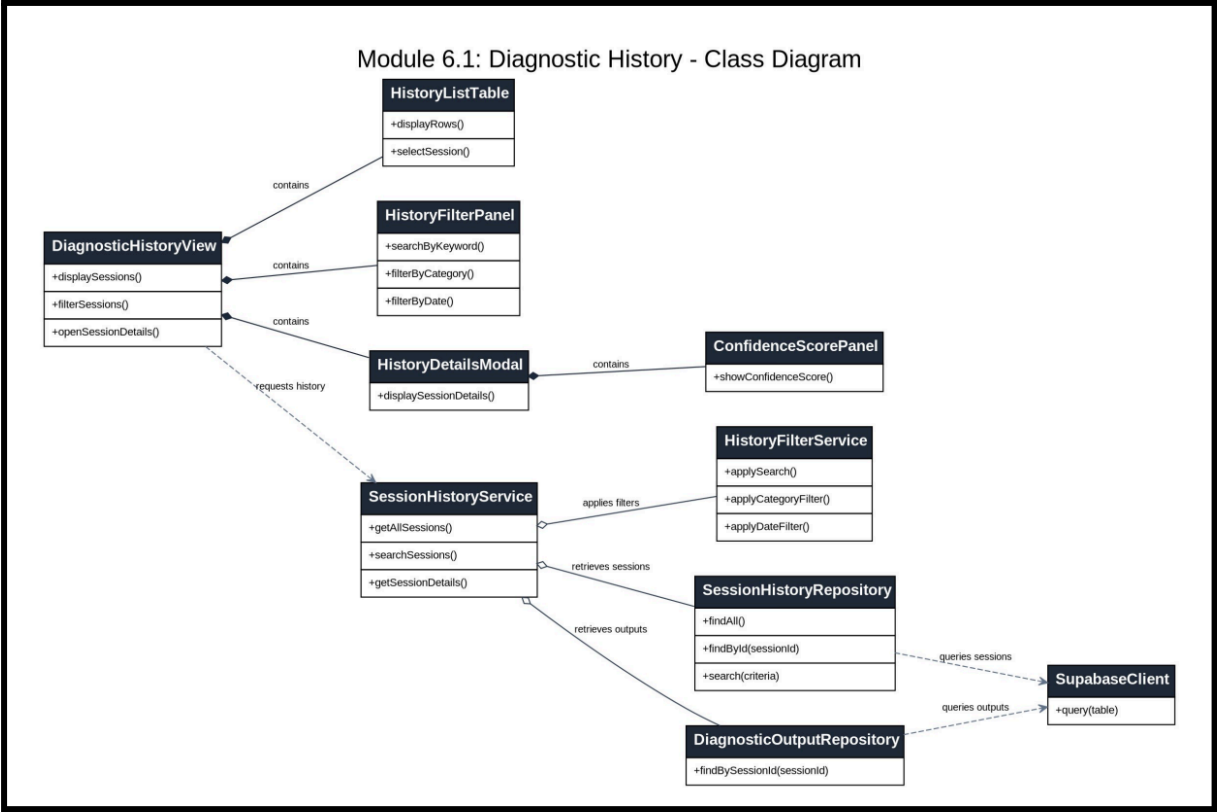
Component Name	Description and Purpose	Component Type / Format
RecurringPatternsView	Displays repeated symptoms, repeated cause categories, occurrence count, and trend labels.	React Page

PatternCard	Shows the main repeated issue detected by the system.	React Component
PatternTrendCard / PatternDots	Displays whether the issue is stable, improving, worsening, or escalating.	React Components
PatternTimelineTable	Lists previous sessions connected to the recurring pattern.	React Table
NoPatternMessagePanel	Displays a message when no recurring pattern is detected.	React Component

Table 3.6.2B. Back-end Components for Recurring Patterns

Component Name	Description and Purpose	Component Type / Format
RecurringPatternService	Detects recurring symptoms and recurring internal cause categories.	C# Application Service
Recurring Pattern Logic	Compares diagnostic history records and calculates trends dynamically.	C# Application Logic
DiagnosticSessionRepository	Provides session history data for pattern analysis.	EF Core Repository
RemediationRepository	Provides remediation outcomes so failed actions can be deprioritized.	EF Core Repository
DiagnosticOutput	Provides cause categories and action outcomes for trend comparison.	C# Domain Entity

Object-Oriented Components



Data Design / ERD or Schema for Recurring Patterns

